




中国研究生创新实践系列大赛
“华为杯”第二十二届中国研究生数学建模竞赛

学　　校		广东工业大学
参赛队号		**25118450079**
队员姓名	**1.**	李洪栈
	2.	刘娴
	3.	周锦涛

中国研究生创新实践系列大赛
“华为杯”第二十二届中国研究生数学建模竞赛

题　目： 基于迁移学习的轴承跨域故障诊断研究

摘　要：

本文针对高速列车**轴承故障诊断**中源域与目标域数据分布不一致的问题，提出了基于**迁移学习**的跨域轴承故障诊断方法。

针对问题一，本文首先综合考虑目标域迁移任务和轴承故障机理，从源域数据中筛选驱动端 (DE) 和风扇端 (FE) 振动数据作为数据集，并对 8 个缺失样本进行填充处理；接着对处理过后的数据集进行**去噪**、**去趋势**、**标准化**等预处理，消除干扰与冗余信息；随后从预处理后的数据中提取**时域**、**频域**、**时频域**共 41 维特征，采用**主成分分析 (PCA)** 降维至 13 维（累计方差解释率达 96.48%），构建了具有高代表性的特征集。

针对问题二，为解决特征集中各故障类型样本数量的不平衡问题，本文采用 **SMOTE 过采样方法**使特征集新增 29,471 条样本，平衡各故障类型样本个数至 8,038 条，并按 7:3 比例划分训练集与测试集。随后通过建立 **XGBoost**、**逻辑回归**、**朴素贝叶斯**、**支持向量机**和 **BP 神经网络**五种诊断模型对测试集进行诊断分析并对比诊断结果，得出结论：XGBoost 模型表现最优，其在测试集中的故障类型诊断**准确率达 0.9991**，**AUC 值为 1.000**，实现了源域的高精度故障诊断。

针对问题三，本文在不改变目标域数据物理属性的情况下，将目标域 16 个样本通过**滑窗采样**扩充至 **3,952** 个，设计了基于 XGBoost 的无约束 (**UTL-ABC**) 和有约束 (**CTL-ABC**) 两种迁移学习模型对目标域样本进行分类标定。两种模型的诊断结果为：UTL-ABC 算法将 MMD 距离从 0.30 降至 0.295，**平均置信度**从 0.38 提升至 0.82，提升幅度达 115.8%。相比之下，CTL-ABC 算法表现更优，MMD 降至 0.29，平均置信度达 0.865，约束违反程度从 7.0 快速降至 1.8 并趋于稳定。两算法预测一致率高达 **92.7%**，CTL-ABC 在 6 项核心指标中均优于 UTL-ABC。最终诊断结果显示，目标域 16 个文件中 12 个具有高可靠性，成功实现了跨域故障分类。

针对问题四，为解决机器学习模型普遍存在的“黑箱”问题，本文构建了三层次的**可解释性**框架，该框架对迁移模型的全过程完整分析结果如下：**事前**可解释性通过 XGBoost 树模型保证结构透明性和特征重要性量化；**迁移过程**可解释性通过 **t-SNE 可视化**展示知识迁移路径，MMD 距离减少 5.2%，置信度提升 113.5%；**事后**可解释性建立诊断结果与故障机理的映射关系，68.8% 的诊断结果具有高可靠性 (置信度 >0.8)。该框架有效提升了模型的可信度和实用性。

关键词： 轴承故障诊断　主成分分析　XGBoost　迁移学习　可解释性

目录

1 问题重述 5

1.1 问题背景 5

1.2 问题提出 6

2 模型假设 7

3 符号说明 7

4 问题一的求解 8

4.1 问题一分析 8

4.2 数据清洗与筛选 9

4.2.1 缺失值处理 9

4.2.2 异常值处理 10

4.2.3 去噪声与去趋势处理 10

4.2.4 标准化处理 11

4.3 特征提取 12

4.3.1 时域特征 12

4.3.2 频域特征 12

4.3.3 时频域特征 13

4.4 滑窗采样处理 13

4.5 特征分析与可视化 15

4.5.1 基于源域的 PCA 特征降维 15

4.5.2 相关性分析与可视化 16

4.6 问题一小结与分析 18

5 问题二的求解 19

5.1 问题二分析 19

5.2 数据集不均衡的分析与处理 20

5.3 基于五种模型的训练和评估 21

5.3.1 XGBoost 21

5.3.2 Logistic Regression 22

5.3.3 Naive Bayes 22

5.3.4 SVM 23

5.3.5 BP Neural Network 23

5.3.6 模型的比较与可视化 23
5.4 问题二小结与分析 25
6 问题三的求解 **26**
6.1 问题三分析 26
6.2 目标域数据集处理 27
6.2.1 源域与目标域数据处理方法对比 27
6.2.2 基于目标域的 PCA 特征降维 28
6.3 UTL-ABC 算法 29
6.3.1 UTL-ABC 算法原理 29
6.3.2 UTL-ABC 参数设置 33
6.3.3 基于 UTL-ABC 结果的可视化分析 33
6.4 CTL-ABC 算法 35
6.4.1 CTL-ABC 算法原理 35
6.4.2 CTL-ABC 参数设置 37
6.4.3 基于 CTL-ABC 结果的可视化分析 38
6.5 基于两种算法的评估实验 39
6.6 问题三小结与分析 41
7 问题四的求解 **42**
7.1 问题四分析 42
7.2 事前可解释性分析 42
7.2.1 模型结构的透明性 42
7.2.2 特征重要性量化 43
7.3 迁移过程可解释性分析 43
7.3.1 知识迁移路径可视化 43
7.3.2 约束机制的量化分析 44
7.4 事后可解释性分析 45
7.4.1 决策依据追溯 45
7.4.2 故障机理关联 45
7.4.3 置信度分析与可靠性评估 45
7.5 综合可解释性框架 45
7.6 问题四小结与分析 46
8 模型评价 **47**
8.1 模型优点 47

8.2 模型缺点 47
附录 A 参考文献 **48**
附录附录 **49**
代码一：UTL-ABC 算法训练.py 49
代码二：CTL-ABC 算法训练.py 65

1 问题重述

1.1 问题背景

高速列车因其安全高效、便捷舒适、绿色低碳等优点，已成为中国客运的主流运输工具；其中，轴承作为走行系统的关键旋转部件，长期处于高转速、交变载荷等复杂恶劣的工况环境中，具有故障率高、易损坏等特点，是设备故障的主要源头。一旦轴承发生故障，轻则导致列车延误和晚点，重则可能诱发脱轨等恶性事故，危及生命安全。特别是在长时间运行过程中，轴承部件易因过热、润滑不良、腐蚀等多种因素发生损坏，进而严重影响列车的正常运行。

当前高铁轴承状态监测方法主要依赖于专家经验构建的特征指标或传统信号处理技术实现。随着轨道交通系统向高密度、强耦合、智能化的方向快速发展，现有方法在诊断精度、泛化能力和实时性方面难以满足复杂运营场景下的精准诊断需求。近年来，随着大数据、人工智能技术的快速发展，数据驱动的智能故障诊断方法 [1] 在列车装备运维领域受到了广泛关注。基于海量运营数据构建的深度学习模型，能够实现更高精度的故障识别、更强的工况适应性和更高效的实时诊断能力。然而，在实际应用场景中，受复杂运行环境与多变工况的影响，传感器采集的原始振动信号易受到背景噪声、干扰源响应等诸多成分影响，这不仅削弱了故障特征的显著性，也直接制约了深度学习模型 [2] 的识别准确率。更关键的是，为保障列车安全运营，关键设备的异常状态通常会被及时处理，导致在途列车的真实故障数据极度稀缺，进而造成训练数据分布严重失衡。

与实际运行数据相对的是，在台架实验环境下采集的轴承数据不仅数量丰富、标签完备，而且其故障演化机理也与在途列车轴承相似。在此背景下，迁移学习技术 [3] 为解决上述问题提供了良好思路。因此，通过解析轴承故障机理对既有试验台数据展开分析，提取代表性的轴承故障特征以构建源域模型，再结合迁移学习技术将诊断知识迁移至实际运营列车数据，可有效缓解样本不平衡问题 [4]。

需要指出的是，可解释性是机器学习领域的重要研究方向之一。由于机器学习模型的“黑箱”问题 [5]，其迁移和诊断过程难以被观测和理解，这可能造成使用者对模型结果的不信任或盲目信任，进而影响诊断模型的应用；因此，迁移诊断可解释性研究 [6] 的核心目标，正是解决迁移学习模型在跨工况、跨设备故障诊断中的透明性问题，提高诊断人员对迁移过程和诊断模型输出的理解和信任度。总体而言，轴承故障诊断作为一项通过监测轴承运行信号（以振动加速度信号为主）、运用数据挖掘、信号处理及机器学习等技术手段来判定轴承状态的技术，从本质上属于模式识别问题——即通过信号处理等技术提取故障特征，再借助机器学习模型实现精准分类，其核心目标在于实现对故障的提前预警与精准诊断，从而避免设备突发性失效，为列车的安全运行提供可靠保障。

1.2 问题提出

根据上述题目背景，利用附件提供的知识及数据集，目标是解决如下问题：

问题 1：数据分析与故障特征提取

请考虑目标域的迁移任务，从提供的源域数据中筛选部分数据组成数据集。结合轴承故障机理，选择合适的方法或指标对有代表性的源域数据进行特征分析，并对整体数据集进行特征提取，用于后续诊断任务。

问题 2：源域故障诊断

在任务 1 提取的故障特征基础上，划分源域训练集与测试集。设计合适的诊断模型实现源域诊断任务，并对诊断结果进行评价。

问题 3：迁移诊断

在任务 2 设计的诊断模型基础上，充分考虑源域与目标域的共性与差异特征，设计合适的迁移学习方法，构建目标域诊断模型，对目标域未知标签的数据进行分类和标定，给出迁移结果的可视化展示和分析，并给出数据对应的标签。

问题 4：迁移诊断的可解释性

可解释性是机器学习领域的重要研究方向之一。由于机器学习模型的“黑箱”问题，其迁移和诊断过程难以被观测和理解，这可能造成使用者对模型结果的不信任或盲目信任，进而影响诊断模型的应用。迁移诊断可解释性研究的核心目标是解决迁移学习模型在跨工况、跨设备故障诊断中的透明性问题，提高诊断人员对迁移过程和诊断模型输出的理解和信任度。请考虑任务 3 中模型的结构设计、迁移过程和决策过程，结合轴承故障特点与故障机理，对迁移诊断的事前/迁移过程/事后（任选一点或多点）可解释性进行分析。

2 模型假设

1. 所有传感器在采集数据期间工作正常，没有因传感器故障或脱落而产生严重失真的数据；
2. 源域数据中的每一个样本只包含外圈、内圈、滚动体和正常运行四种中的一种状态，不存在多种状态耦合的情况；
3. 源域数据集中的所有样本的类别标签（故障类型、位置、尺寸等）都是准确无误的；
4. 尽管工况（转速、负载、环境）不同，但源域和目标域中，由同类轴承故障引起的振动信号核心物理机理是相似的。

3 符号说明

符号	意义
μ	序列的算术平均值
σ	序列的标准差
$x(n)$	振动信号，(n=1,2,...,N), N 为信号长度
$X(k)$	信号傅里叶变换
f_k	故障频率
$S(k)$	功率谱密度
rpm	轴承转速
N_b	滚动体数量
d	滚动体直径
D	轴承节径
$w(t-\tau)$	窗口函数
T_i	相应的故障周期 (i=O,I,B)
F_{time}	时域特征
F_{freq}	频域特征
F_{tf}	时频域特征
α, β, γ	权重系数
rand$(0,1)$	随机插值因子，为 0 ～ 1 均匀分布的随机数
$(\widetilde{x}-x)$	特征差异向量
$\phi(\cdot)$	映射到再生希尔伯特空间的特征映射函数
$\|\cdot\|_{\mathcal{H}}$	再生希尔伯特空间中的范数

4 问题一的求解

4.1 问题一分析

考虑目标域的迁移任务，从提供的源域数据中筛选出合适的数据集，结合轴承故障机理，提取具有代表性的源域数据进行特征分析，并对整体数据集进行特征提取，为后续迁移诊断任务做准备。

本问题的核心工作为两个关键步骤：其一，综合目标域迁移任务和故障机理筛选源域数据，并对代表性数据进行特征分析；其二，对整体数据进行特征提取，从而为迁移任务奠定基础。根据以上条件，问题一的解答流程图如图 22所示。详细步骤如下：

第一步：筛选源域数据集。依据迁移学习目标和故障机理，筛选驱动端 (DE) 和风扇端 (FE) 的振动数据作为数据集，并给出合理的解释。

第二步：确定代表性数据集的分析指标。根据故障机理，确定分析指标为时域、频域、时频域三方面的特征。

第三步：对代表性数据集进行特征分析。根据第二步选定的指标，结合故障机理对代表性数据集进行特征分析与讨论。

第四步：对整体数据集进行特征提取。结合一至三步，对整体数据集特征提取后，介绍相关特征并为后续迁移学习做准备。

问题一的解题步骤流程如图 1 所示。

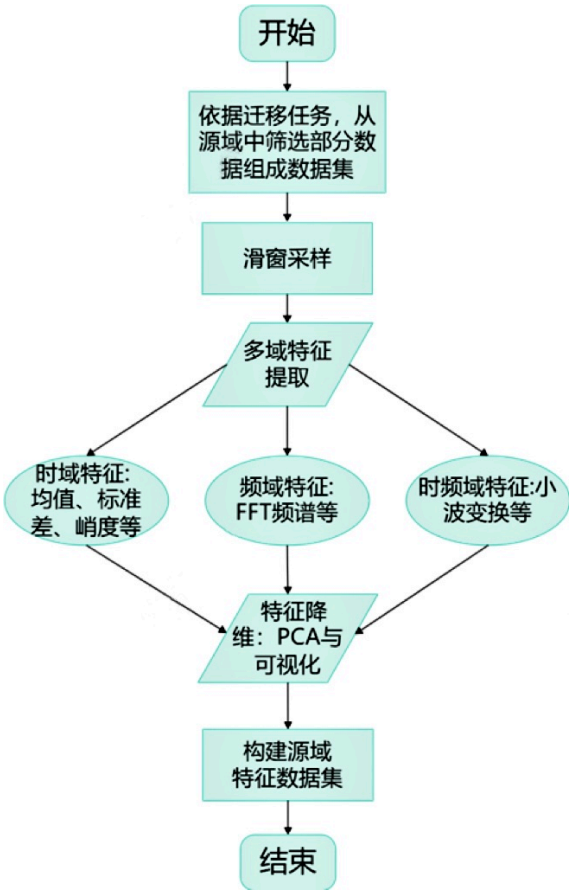


图 1　问题一技术路线图

为保证迁移学习模型和故障诊断模型的有效性和可解释性。本文选择源域数据中的驱动端 (DE) 和风扇端 (FE) 的振动数据作为数据集。该决策基于以下两方面考量：

1. 基于迁移学习目标：迁移学习的核心前提是源域和目标域之间存在共同的特征空间。且迁移学习模型应适用于轴承类故障的通用模式，而不是聚焦于特定机器的结构特性。驱动端数据 (DE) 与风扇端数据 (FE) 都是直接反映轴承振动的故障共性，受特定机器结构的影响较小，故障特征可迁移到不同电机结构的目标域中。而基座端采集的振动信号受到电机整体结构、电机运行状态、地基等影响，虽反映电机整体的振动信号，但经过多层结构的传递，其信号故障特征已高度衰减，不适合用于故障特征的提取。若考虑源域中基座端数据 (BA) 来建立迁移模型，其特征无法迁移到不同地基、不同底座结构的电机振动数据中，且易出现负迁移。

2. 基于故障机理：驱动端和风扇端都直接连接相应的转轴，均能清晰捕获相应端轴承的振动信号和经过转轴传递后另一个端的振动信号，信号的信噪比高。而基座数据 (BA) 是经过基座、地基等结构传递和滤波后的结果，其信号强度在传递过程中严重衰减，可能导致故障特征不显著。

综上所述，选择驱动端数据和风扇端数据，可有效剥离电机特定结构带来的干扰因素，确保了后续提取数据特征的代表性和可迁移性，也提高后续建立的迁移学习模型的有效性和鲁棒性。

4.2 数据清洗与筛选

为消除数据中的干扰成分，提升信号质量，并为后续特征提取奠定基础，本研究对所有样本数据的驱动端（DE）与风扇端（FE）振动信号进行了系统的预处理。流程主要包括缺失值处理、异常值检测与处理、去趋势以及去噪四个核心步骤。

4.2.1 缺失值处理

在对数据集筛选的过程中，本文发现有 8 个样本缺失了在风扇端采样信号的数据，即 FE_time 的数据，具体缺失的数据如下表展示：

表 1　FE_time 缺失值样本列表

序号	样本名称	序号	样本名称
1	12kHz-DE-B-0028-0	5	12kHz-DE-IR-0028-0
2	12kHz-DE-B-0028-1	6	12kHz-DE-IR-0028-1
3	12kHz-DE-B-0028-2	7	12kHz-DE-IR-0028-2
4	12kHz-DE-B-0028-3	8	12kHz-DE-IR-0028-3

表 1 中的数据展示了缺失的数据主要是在 12kHz 下驱动端的滚动体故障以及内圈故障的样本，且都为 0.028 尺寸在不同负荷的样本，考虑到若选取相同工况和运行状态的同类型端口的振动数据求和取均值填充缺失值，会破坏原样本中三个端数据的时间相关性，易丢失真实振动信号应有的波形与数据特征。

因此，为保证填充后数据集依然保留真实振动信号应有的时序和频域特征，本文采用基于相似性的样本替换的方法来填充缺失值。具体流程如下：

Step1：汇总与缺失样本同工况、马力、故障类型且 FE 端数据完整的样本。

Step2：通过计算相应完整样本中 DE 端数据与缺失样本中 DE 端数据的欧式距离来确定各样本之间的相似度。

Step3：优先选择 DE 端数据相似度最高的完整样本中的 FE 端数据对缺失样本进行填充。

4.2.2 异常值处理

对于填充后的完整数据集，本文对每个样本数据的各端口的采样信号采用箱线图 (Box-plot) 自动识别数据集中的异常值，若某个样本中的某个端的采样信号数据存在异常值点，那么利用该异常值点相邻的信号数据取均值进行替换。鉴于数据集的规模，文中难以展示全部样本的检测与处理结果，本文随机选取了 1 个具有代表性的样本数据进行展示，如图 2 所示。

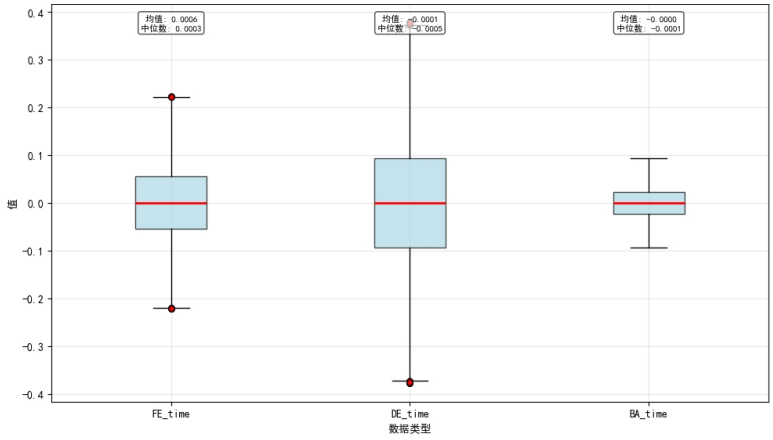


图 2　样本编号为 12kHz-DE-B007_1 的箱线图

图 2 展示的是在 12kHz 下驱动端下的驱动端、风扇端以及基座采样信号的箱线图，图中可知：DE_time 离散程度最高，FE_time 中等，BA_time 最低；FE_time 与 DE_time 都存在上下双侧异常值，BA_time 无显著异常值；三者均呈现不同程度的右偏。

4.2.3 去噪声与去趋势处理

本文采用线性去趋势与移动平均滤波相结合的方法对数据集进行处理，以消除趋势成分并抑制高频噪声。

在去趋势处理中，本文采用线性去趋势算法来消除数据集中可能存在的线性漂移趋势。该算法包括：趋势拟合、计算趋势分量和趋势消除三部分，本文的数据处理通过 Python 来实现，确保了计算的效率和准确性。

在噪声抑制方面，本文采用中心移动平均滤波算法来抑制数据集中的高频噪声成分。该方法能有效平滑高频噪声，保留原数据集的主要特征。算法中具体参数设置如下：

1. 滤波窗口：采用固定窗口大小为 5 个数据点的滑动平均滤波器。窗口大小强制设为奇数，即当输入为偶数时，自动加 1。

2. 滤波原理：对于数据集中每个数据点 x_i，以窗口内相邻点的算术平均作为去噪后的值。

$$y_i = \frac{1}{w} \sum_{j=-k}^{k} x_{i+j} \tag{4.1}$$

其中 ω 为窗口大小，k 为半窗宽度。

3. 可视化分析：去噪声与去趋势处理的前后对比图如图 3所示。

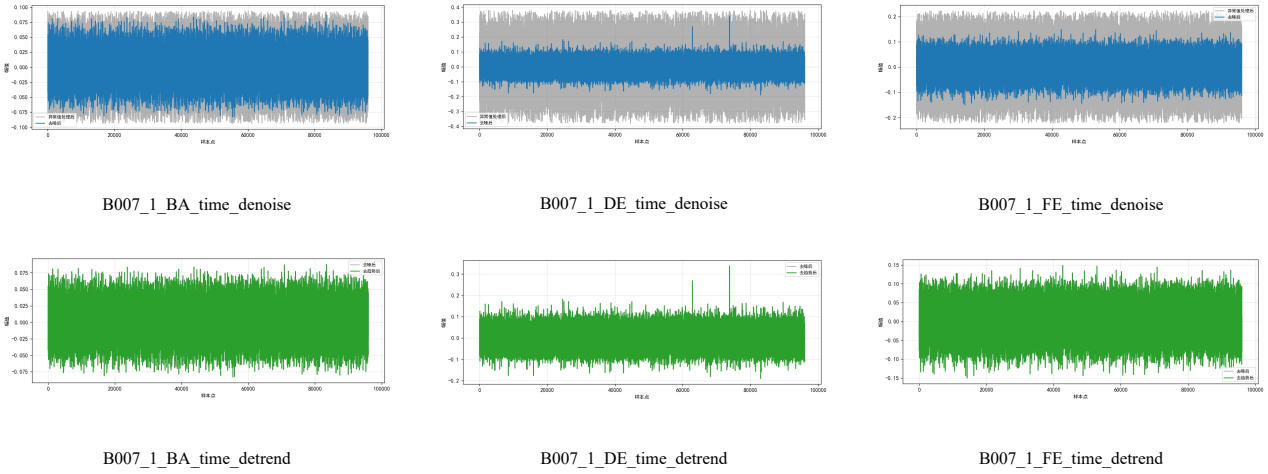


图 3　去噪声与去趋势的可视化

上图展示信号预处理（去噪 + 去趋势）前后对比，本文发现处理效果显著，经过预处理后，原始信号中的高频随机噪声被有效抑制，同时信号中存在的缓慢变化趋势项也成功被移除了。最重要的是，处理过程较好地保留了代表轴承故障特征的周期性冲击成分，这说明了预处理在降噪与去趋势的过程中没有损失掉关键的诊断信息。该图证实了所采用的处理方法能够为后续的故障诊断任务提供高质量的输入数据，这是保证模型准确性的关键一步。

4.2.4 标准化处理

为消除 DE 与 FE 中振动数据因轴承量纲和采集条件的差异导致的数据尺度不统一问题，本文采用 Z-score 标准化方法对数据进行标准化化处理，以消除不同特征之间的量纲和数量级影响。

$$x_{\text{standard}} = \frac{x - \mu}{\sigma} \tag{4.2}$$

其中，x 为原始振动幅值，μ 为序列的算术平均值：$\mu = \frac{1}{n}\sum_{i=1}^{n} x_i$，$\sigma$ 为序列的标准差：$\sigma = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(x_i - \mu)^2}$，$x_{\text{standard}}$ 为标准化后的结果，并且服从均值为 0、标准差为 1 的分布。

4.3 特征提取

轴承故障信号本身是一个非平稳、非线性的复杂信号，且其具有周期性冲击特性，单一分析维度无法准确捕捉信号的全部信息。因此本文采用时域、频域、时频域特征三维度相结合的方法来提取中源域数据故障特征。

4.3.1 时域特征

在时域分析中，本文提取均值、标准差、峭度等特征指标来捕捉故障信号的波形统计特征和形态变化。

时域分析中部分指标的计算公式如下：

均值

$$\mu = \frac{1}{N}\sum_{n=1}^{N} x(n) \tag{4.3}$$

标准差

$$\sigma = \sqrt{\frac{1}{N-1}\sum_{n=1}^{N}[x(n)-\mu]^2} \tag{4.4}$$

峭度

$$K = \frac{\frac{1}{N}\sum_{n=1}^{N}[x(n)-\mu]^4}{\sigma^4} - 3 \tag{4.5}$$

其中，$x(n)$ 为振动信号，$(n = 1, 2, ..., N)$，N 为信号长度。

在频域分析中，本文将故障信号数据通过傅里叶变换转化为故障频率，提取故障特征频率以及倍频的幅值等指标，来反映故障信号的周期成分和频域能量的分布模式。

4.3.2 频域特征

频域分析中部分频谱统计特征和故障特征频率的计算公式如下：

频谱质心

$$f_c = \frac{\sum_{k=1}^{N/2} f_k \cdot S(k)}{\sum_{k=1}^{N/2} S(k)} \tag{4.6}$$

外圈故障特征频率 (BPFO)

$$BPFO = f_r \cdot \frac{N_d}{2} \cdot \left(1 - \frac{d}{D}\right) \tag{4.7}$$

内圈故障特征评率 (BPFI)

$$BPFI = f_r \cdot \frac{N_d}{2} \cdot \left(1 + \frac{d}{D}\right) \tag{4.8}$$

滚动体故障特征频率 (BSF)

$$BSF = f_r \cdot \frac{D}{d}\left[1 - \left(\frac{d}{D}\right)^2\right] \tag{4.9}$$

其中，$X(k)$ 为信号的傅里叶变换，f_k 为频率，$S(k) = |X(k)|^2$ 为功率谱密度。$f_r = \frac{n}{60}$ 为转频 (Hz)，n 为轴承转速（rpm），N_b 为滚动体数量，d 为滚动体直径，D 为轴承节径。

4.3.3 时频域特征

时频域特征部分指标的计算公式如下：

连续小波变换公式 (CWT)

$$W_x(a,b) = \frac{1}{\sqrt{a}} \int_{-\infty}^{\infty} x(t)\psi\left(\frac{t-b}{a}\right) dt \tag{4.10}$$

其中，$W_x(a,b)$ 为信号，$x(t)$ 的尺度平移小波变换，ψ 为母小波，a 和 b 分别为尺度和平移参数。

短时傅里叶变换

$$STFT\{x(t)\} = \int_{-\infty}^{\infty} x(t) \cdot w(t-\tau) \cdot e^{-j2\pi ft} dt \tag{4.11}$$

其中，$w(t-\tau)$ 是窗口函数，$x(t)$ 是信号，f 是频率，τ 是时间。

时频复合特征

$$F_{\text{composite}} = \alpha \cdot F_{\text{time}} + \beta \cdot F_{\text{freq}} + \gamma \cdot F_{tf} \tag{4.12}$$

其中 F_{time}、F_{freq}、F_{tf} 分别为时域、频域、时频域特征，α、β、γ 为各特征的权重系数。

4.4 滑窗采样处理

为了充分利用振动信号特征并增加训练样本的数量，本文采用滑动窗口采样方法对源域数据集进行分段处理。滑窗采样法不仅能够捕捉信号的局部动态特性，还能有效扩充数据集规模，为后续的特征提取和模型训练提供充足的样本支撑。

（1）滑窗采样原理

滑动窗口采样是一种时间序列数据增强技术，其基本思想是使用固定大小的窗口在原始信号上滑动，每次移动固定的步长，从而获得多个重叠或非重叠的子序列。对于长度为 N 的原始信号 $x(n)$，使用窗口大小 W 和步长 S 进行采样，可获得的窗口数量为：

$$N_{windows} = \left\lfloor \frac{N-W}{S} \right\rfloor + 1 \tag{4.13}$$

其中，$\lfloor \cdot \rfloor$ 表示向下取整。第 i 个窗口的数据段可表示为：

$$x_i(n) = x(n+iS), \quad n = 0,1,...,W-1 \tag{4.14}$$

滑窗采样的示意图如图4所示。

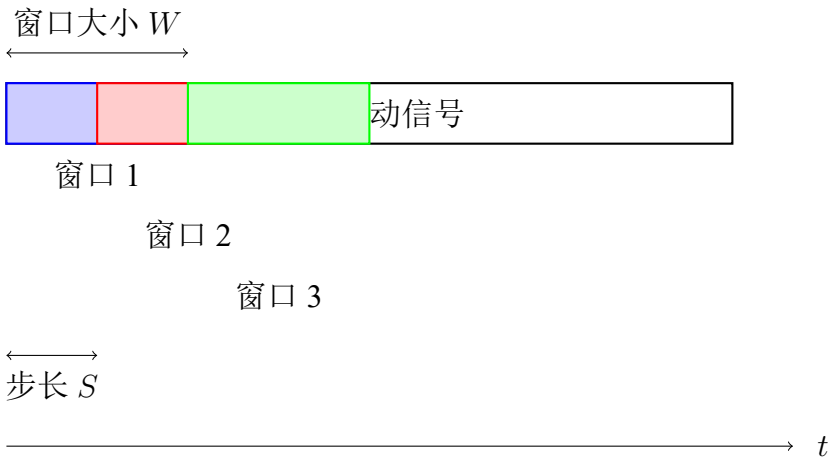


图 4　滑动窗口采样示意图

（2）基于故障机理的参数选择

本文的创新点在于根据不同故障类型的物理机理自适应地选择窗口大小和步长，而非采用固定的参数。这种方法能够更好地捕捉不同故障的周期性特征。

根据轴承故障机理，不同故障类型具有不同的特征周期：

- **外圈故障（OR）**：故障特征以转动周期 T_r 为主要调制周期，故障冲击以 T_{BPFO} 为间隔周期出现。
- **内圈故障（IR）**：故障特征受转频调制，主周期为 T_r，故障冲击周期为 T_{BPFI}。
- **滚动体故障（B）**：故障特征受滚动体公转频率调制，主周期为 T_{FTF}，故障冲击周期为 T_{BSF}。

基于上述分析，本文设计了自适应的窗口参数选择策略：

$$(W, S) = \begin{cases} (T_r \cdot f_s, T_{BPFO} \cdot f_s) & \text{外圈故障} \\ (T_r \cdot f_s, T_{BPFI} \cdot f_s) & \text{内圈故障} \\ (T_{FTF} \cdot f_s, T_{BSF} \cdot f_s) & \text{滚动体故障} \\ (\text{rand}(0.01, 0.09) \cdot f_s, \text{rand}(0.001, 0.009) \cdot f_s) & \text{正常状态} \end{cases} \tag{4.15}$$

其中，f_s 为采样频率（12kHz 或 48kHz），各故障周期的计算公式为：

$$T_{BPFO} = \frac{1}{BPFO} = \frac{120}{n \cdot N_d \cdot (1 - \frac{d}{D})} \tag{4.16}$$

$$T_{BPFI} = \frac{1}{BPFI} = \frac{120}{n \cdot N_d \cdot (1 + \frac{d}{D})} \tag{4.17}$$

$$T_{BSF} = \frac{1}{BSF} = \frac{60d}{n \cdot D \cdot [1 - (\frac{d}{D})^2]} \tag{4.18}$$

$$T_{FTF} = \frac{1}{FTF} = \frac{120}{n \cdot (1 - \frac{d}{D})} \tag{4.19}$$

其中，上面的 n、d 和 D 的参数值的确定由下表 2 所示：

表 2　源域数据集轴承尺寸参数

轴承类型	滚动体数 n	滚动体直径 d	轴承节径 D
SKF6205 (DE)	9	0.3126 英寸	1.537 英寸
SKF6203 (FE)	9	0.2656 英寸	1.122 英寸

4.5 特征分析与可视化

综合上述内容，本文在经过处理后的 DE 和 FE 数据集中，共提取了 41 个时域、频域和时频域的特征。这些特征均能全面的反映轴承的运行状态，但高维度的空间特征可能存在空间多重共线性，导致信息冗余。且对于后续目标域的诊断任务，高维度的特征空间会大大增加计算复杂度和存储需求。因此，本文采用主成分分析法 (PCA) 对数据集进行特征降维，以提高后续机器学习模型的训练效率。

4.5.1 基于源域的 PCA 特征降维

在进行主成分分析之前，本文对数据集进行了系统的预处理，包括以下步骤：

1. 轴承故障频率相关特征 (BSF、BPFI、BPFO 等) 共有 12 处存在缺失值的特征列，统一采用零值填充。

2. 对位置 (position) 和载荷 (Load) 的分类变量进行独热编码，生成 9 个新的二进制特征。

3. 将处理后的数据及维度从 48 维扩展到了 55 维，经过有效的特征选择（剔除重复的特征标签、剔除与故障诊断无关的特征标签）后，特征空间最终为 41 维。

主成分分析的降维过程如下：首先，为消除数据集特征量纲的影响，本文采用 Z-score 标准化方法使每个特征的均值为 0、方差为 1。接着，采用主成分分析法对标准化后的数据进行分析，计算其特征的协方差矩阵，并分解其特征值和特征向量。

最后，根据对方差的分析，发现 Top 13 主成分累计可解释 96.48% 的总体方差, 依据广泛使用的 95% 方差准则，因此我们了选取前 13 个主成分作为有效特征。Top 13 主成分的方差结果如表 3 所示。

表 3　主成分方差分析结果

主成分	方差贡献率 (%)	累计方差解释率 (%)
PC1	22.79	22.79
PC2	19.65	42.45
PC3	14.68	57.13
PC4	9.67	66.80
PC5	5.80	72.60
PC6	4.42	77.02
PC7	3.30	80.32
PC8	3.17	83.49
PC9	3.11	86.60
PC10	2.78	89.38
PC11	2.45	91.83
PC12	2.23	94.06
PC13	2.42	96.48

4.5.2 相关性分析与可视化

（一）基于 PCA（主成分分析）绘制了 Top 20 主成分方差解释的碎石图，用于分析降维后各主成分的重要性, 如图 5所示。

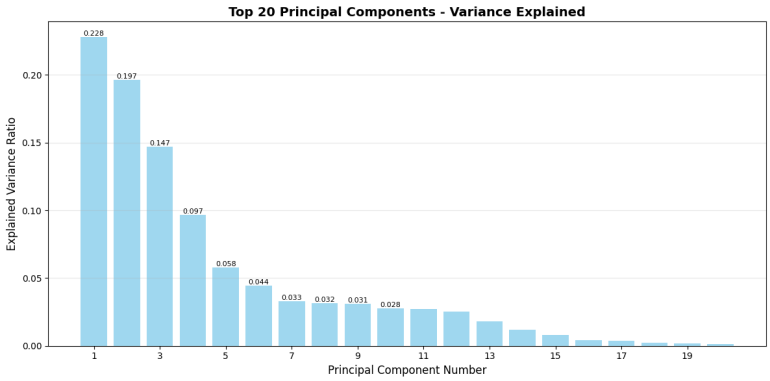


图 5　Top 20 特征的方差解释碎石图

上图中展示了前 20 个主成分的方差贡献率，其中 PC1 的方差解释度约为 22.8%，PC2 方差解释度约为 19%。前几个主成分能够解释相当比例的方差，说明提取的时频域特征有效。其中若保留前 10 个主成分，累计方差几乎接近 90%。

该碎石图表明轴承故障的特征具有较好的可降维性，但需要保留 4-5 个主成分以上才能在故障诊断中取得理想效果，但在实际建模时需要根据具体诊断任务调整降维的维度。

（二）基于 PCA（主成分分析）降维绘制了基于前两个主成分的特征空间分布图，是评估故障特征可分性的重要工具之一, 如图 6所示。

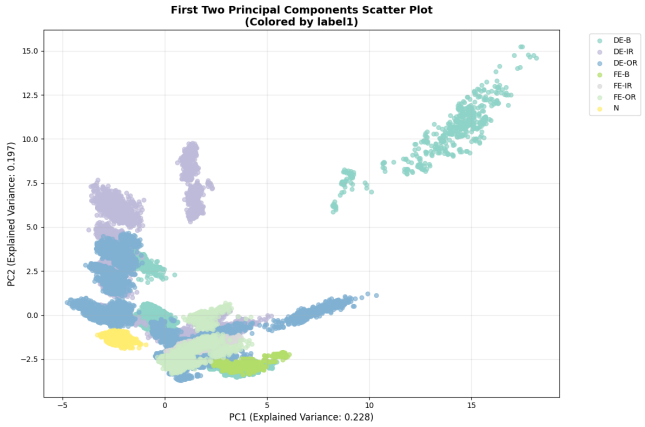


图 6　前两个主成分的散点图

上图中展示了经过 PCA 降维后最重要的两个维度（主成分 1 和主成分 2）的故障特征分布, 发现除了 DE-B 类型的故障簇相对远离，而其他相同故障类型的数据点都相对集中，表明了轴承故障的特征具有较好的可降维性。

（三）基于 PCA（主成分分析）降维绘制了主成分-特征载荷热力图，是分析特征之间相关性的有效工具, 如图 7所示。

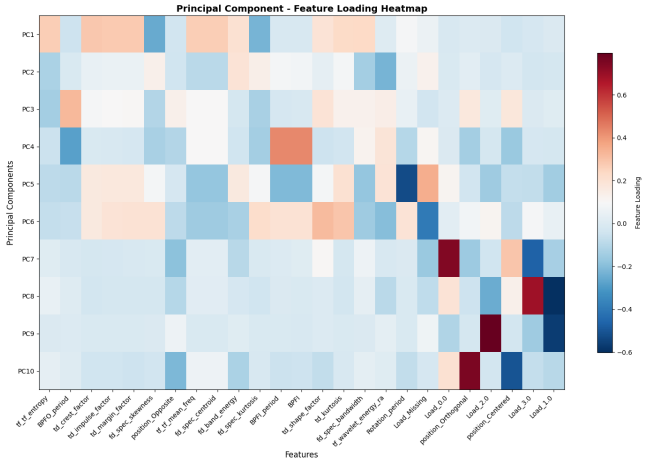


图 7　主成分-特征载荷热力图

图中展示了主成分与各个特征之间的相关性，其中深红色（+1.0）为完全正相关，深蓝色（-1.0）为完全负相关，白色（0.0）为无相关性。我们发现 PC 10 与 position_Orthogonal，PC 9 与 Load_2.0,PC 7 与 Load_0.0 为完全正相关,PC 8 与 Load_1.0 为完全负相关等。同时，对于高正相关的特征组，我们可以保留其中一个，避免多重共线性。通过对热力图分析，我们可以构建一个特征间适度相关、且与目标高度相关、无冗余的特征集合，为后续故障诊断模型奠定基础。

4.6 问题一小结与分析

针对问题一，首先本文对数据进行了预处理，并对源域数据集进行了筛选；其次，本文采用了 PCA 方法实现了降维，发现 Top 13 主成分累计可解释的总体方差达到了 96.48%，因此选取了 TOP 13 个主成分作为有效特征；同时本文并对特征相关性进行了分析与可视化；最终，对整体数据集进行了一个低冗余、高代表性的特征集合的提取，用于后续诊断任务。

5 问题二的求解

5.1 问题二分析

本问题的核心工作为两个关键步骤：其一，是在问题一提取的故障特征基础上，划分源域为训练集与测试集。其二，是设计合适的故障诊断模型实现源域诊断任务，并对诊断结果进行评价。

针对第一小问，本文在问题一提取的特征基础上，通过过采样方法平衡数据集后，按照7：3 的比例划分源域数据集为训练集与测试集。

针对第二小问，本文建立了 XGBoost、Logistic Regression、Naive Bayes、SVM 和 BP Neural Network 五种训练模型对源域数据进行故障诊断，并对诊断结果进行了系统性的分析与评价。

问题二的解题步骤流程如图 2 所示。

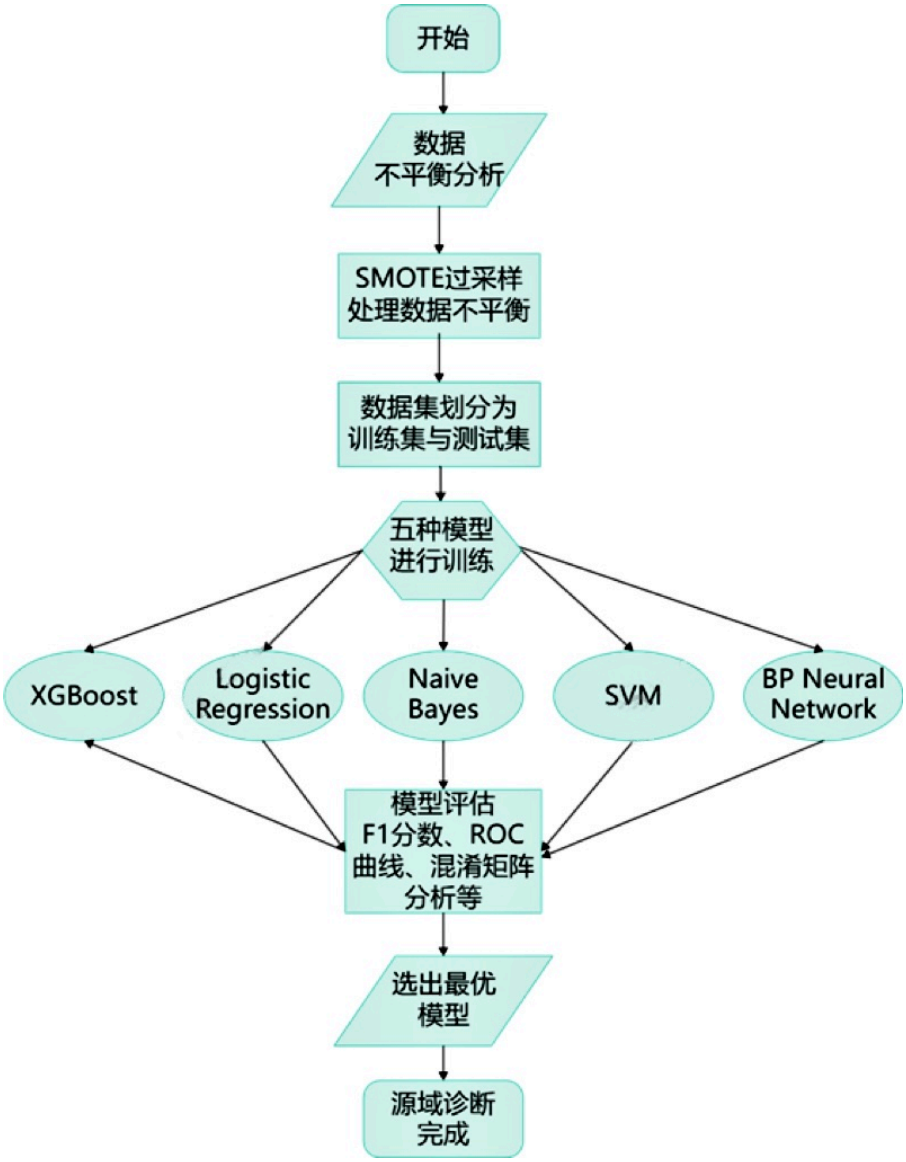


图 8　问题二技术路线图

5.2 数据集不均衡的分析与处理

根据问题一主成分分析降维后得到的数据集,各样本均包含 13 个特征标签,是一个 26795*13 的特征矩阵。特征矩阵中划分了七种故障类型，且各故障类型包含的样本数如表 8所示。

表 4　源域数据故障类型及对应特征标签

故障类型	包含样本个数 (%)	占比 (%)
驱动端外圈故障 (DE-OR)	8038	30
驱动端内圈故障 (DE-IR)	5767	21.52
驱动端滚珠故障 (DE-B)	4997	18.65
风扇端外圈故障 (DE-OR)	2929	10.93
风扇端内圈故障 (DE-IR)	2004	7.48
风扇端滚珠故障 (DE-B)	1826	6.81
正常状态 (N)	1234	4.61

根据表 4 可知，各故障类型的样本数据是不平衡的，为降低数据不平衡问题对机器学习模型产生负面影响，本文以 DE-OR 的样本个数 8038 为标准，优先采用 SMOTE 方法对数据集进行过采样，以均衡各故障样本数量。

SMOTE 方法的步骤如下：

Step1. 筛选样本数少于 8038 的故障类型，并确定每个类别需新生成的样本个数。

Step2. 在 13 维特征空间中找到每个样本的 k 个邻居 (默认为 5)，计算样本间距离。

Step3. 通过线性插值法在样本和邻居之间合成新样本，并合并到原始数据集中。

-SMOTE 算法如下：

$$x_{\text{new}} = x + \text{rand}(0,1) \times (\tilde{x} - x) \tag{5.20}$$

其中，rand(0,1) 为随机插值因子，$(\tilde{x} - x)$ 为特征差异向量。

通过上述过采样方法，新增了 29471 条样本，使得各故障类型的样本数均达到了 8038 条。各故障类型样本数在过采样前后的变化如图 9所示。

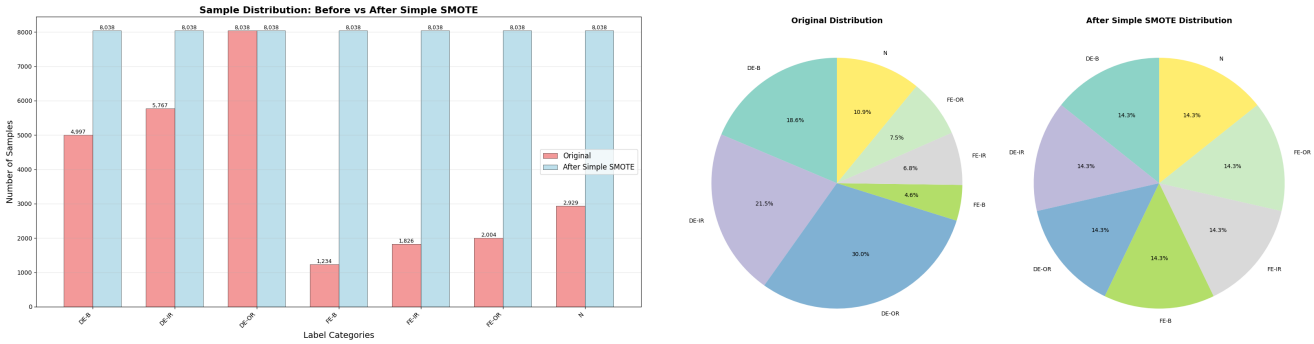


图 9　SMOTE 过采样前后样本分布对比图

从图 9 柱状图可见，经过 SMOTE 处理后，所有故障类别的样本数量都被调整到 8038 个，成功消除了原域数据中因故障类别不同导致的样本个数不平衡问题。而饼状图则进一步验证了采样前后对比分布的均衡性：原始分布中，而 DE-OR 占比达 30.0%，而 FE-B 仅占 4.6%，各故障类别占比差异显著；经过 SMOTE 处理后，所有类别占比趋于一致，均稳定在约 14.3%，实现了故障类别占比的均匀化。对此，本文以 7：3 的比例划分平衡后的源域数据集为训练集和测试集，用于后续源域故障诊断模型的研究。

5.3 基于五种模型的训练和评估

本文选取五种具有代表性的分类模型进行训练，包括：XGBoost、逻辑回归 (Logistic Regression)、朴素贝叶斯 (Naive Bayes)、支持向量机 (SVM) 以及 BP 神经网络。各模型均在前文划分的训练集上进行训练，并通过相同的测试集评估其诊断效能，从而综合比较各算法在复杂故障模式识别中的适应性与局限性。

5.3.1 XGBoost

XGBoost 是一种在决策树基础上优化的模型，采用了集成学习的思想，通过将多个决策树组合在一起，每加入一个决策树，模型的预测效果就会有所提升。XGBoost 属于梯度提升模型范畴，通过不断拟合前面模型的偏差，从而不断降低加法模型的偏差，使得预测值更接近真实值。

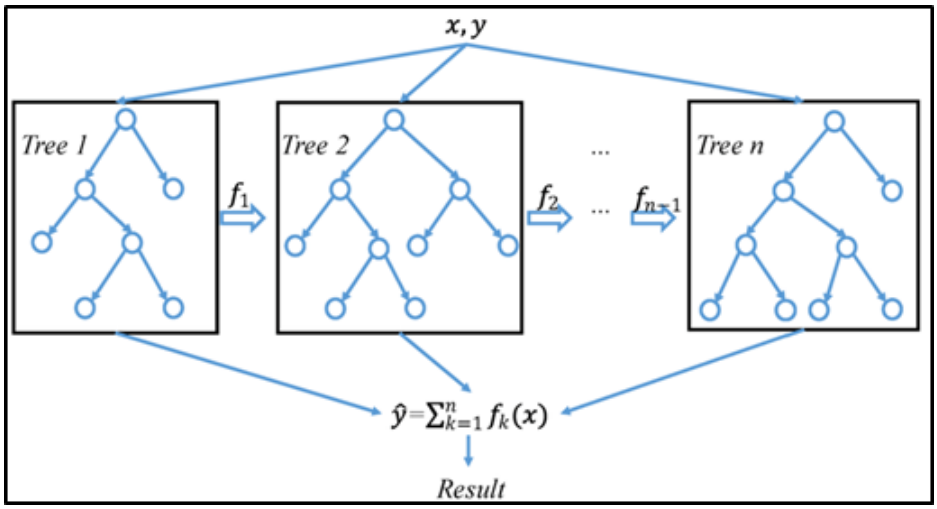


图 10　XGBoost 原理图

XGBoost 分类的具体算法步骤如下：

第一步，决策树的加法模型：

$$\hat{y}_i = \sum_{k=1}^{K} f_k(x_i), \quad f_k \in \mathcal{F} \tag{5.21}$$

其中，$\hat{y}_i$ 表示第 i 个样本的预测值，f_k 表示第 k 棵树的预测函数，$\mathcal{F}$ 是所有树的集合。

第二步，定义优化目标函数，即损失函数：

$$Obj^{(t)} = \sum_{i=1}^{n} l(y_i, \hat{y}_i^{(t-1)} + f_t(x_i)) + \sum_{k=1}^{t} \Omega(f_k) \tag{5.22}$$

其中，l 是损失函数，$\hat{y}_i^{(t-1)}$ 是第 $t-1$ 次迭代的预测值，$f_t(x_i)$ 是第 t 棵树的预测值，$\Omega(f_k)$ 是正则项，用于控制模型的复杂度，防止过拟合。正则项的具体公式为：

$$\Omega(f_t) = \gamma T + \frac{1}{2}\lambda \sum_{j=1}^{T} w_j^2 \tag{5.23}$$

其中，T 是树的叶子数，γ 和 λ 是正则化参数，w_j 是叶子节点的权重。

第三步，使用泰勒展开式对目标函数进行近似：

$$Obj^{(t)} \approx \sum_{i=1}^{n} \left[g_i f_t(x_i) + \frac{1}{2} h_i f_t(x_i)^2 \right] + \Omega(f_t) \tag{5.24}$$

其中，g_i 是损失函数的一阶导数，h_i 是损失函数的二阶导数。

第四步，对每个叶子节点的权重进行优化，最终目标函数变为：

$$Obj = \sum_{j=1}^{T} \left[\left(\sum_{i \in I_j} g_i \right) w_j + \frac{1}{2} \left(\sum_{i \in I_j} h_i + \lambda \right) w_j^2 \right] + \gamma T \tag{5.25}$$

其中，I_j 表示第 j 个叶子节点上的样本集合。

最后一步，通过对目标函数求导并令其等于零，可以得到每个叶子节点的最优权重 w_j：

$$w_j = -\frac{\sum_{i \in I_j} g_i}{\sum_{i \in I_j} h_i + \lambda} \tag{5.26}$$

代入上述结果，目标函数的最小值为：

$$Obj = -\frac{1}{2} \sum_{j=1}^{T} \frac{\left(\sum_{i \in I_j} g_i \right)^2}{\sum_{i \in I_j} h_i + \lambda} + \gamma T \tag{5.27}$$

通过上述步骤，XGBoost 能够高效地进行模型训练和预测，且通过加入正则项，可以有效防止模型的过拟合。

5.3.2 Logistic Regression

逻辑回归（Logistic Regression）是一种概率型线性分类模型。它通过 Sigmoid 函数将线性回归的输出映射到 (0,1) 区间，将其解释为属于某个类别的概率。逻辑回归模型的优点有模型简单，计算成本低，并且训练速度快。该模型的输出具有概率意义，可以直观地判断分类的置信度。

5.3.3 Naive Bayes

朴素贝叶斯（Naive Bayes）基于贝叶斯定理，并假设特征之间是条件独立的（这就是“朴素”的由来）。在给定特征下，样本属于各个类别的后验概率，并选取概率最大的类别。

5.3.4 SVM

支持向量机（SVM）的核心思想是寻找一个最优超平面（在特征空间中的一条线或者面），目标是使得不同类别样本与样本之间的间隔最大化。对于线性不可分的数据，通过“核技巧”映射到高维空间使其线性可分。

5.3.5 BP Neural Network

BP 神经网络（BP Neural Network）是一种多层前馈神经网络，使用误差反向传播算法进行训练。信号前向传播得到输出，反向传播通过调整网络中的连接权重，不断迭代使网络输出逐渐逼近真实的标签。

5.3.6 模型的比较与可视化

（一）为评估不同模型的诊断性能，本文对比了五种算法的关键指标，可视化结果如图 11 所示。

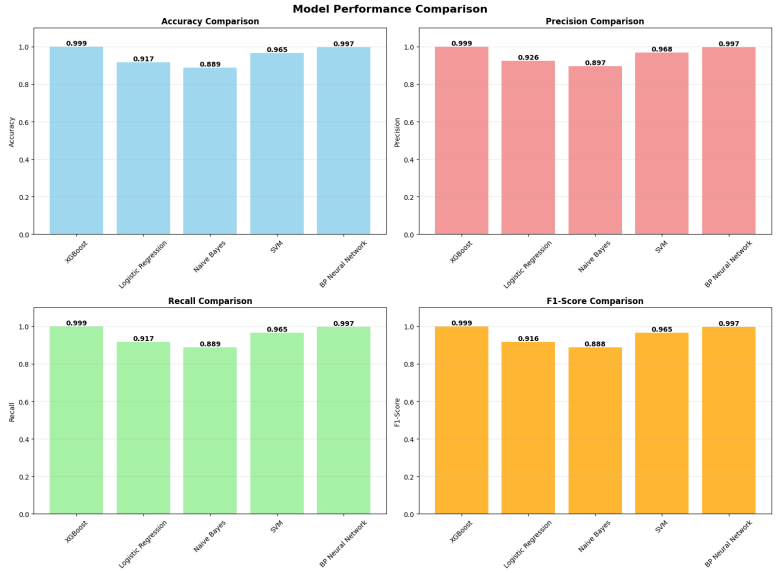


图 11　五种模型性能对比图

上图为多模型对比柱状图，其中分别计算了 F1 分数、准确率、召回率、精确率四个指标。本文发现 XGBoost 四个指标均为 0.999，其性能最优；而 Naive Bayes 的四个指标的值均最低，其表现良好但稍逊。整体来看，五种模型的性能表现都取得了不错的效果。

（二）同时本文绘制了 ROC 曲线比较图评估不同模型的诊断性能，如图 12所示。

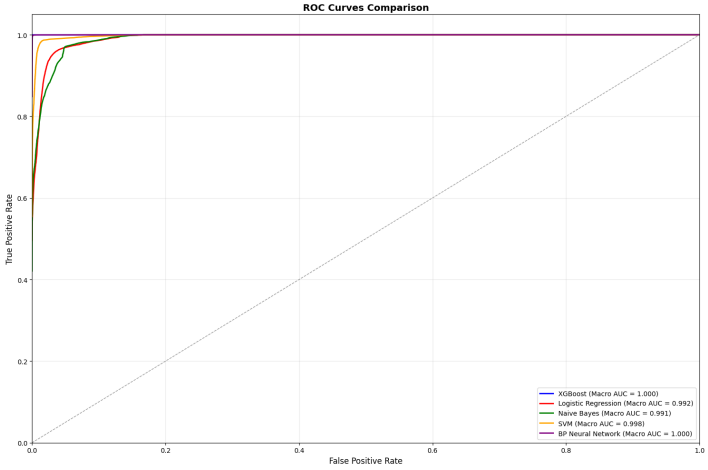


图 12　ROC 曲线比较图

上图为多分类 ROC 曲线比较图，展示了不同模型在轴承故障诊断任务上的性能。其中曲线下面积 (AUC) 越大，则模型的整体性能就越好。本文发现所有模型的曲线都接近左上角，说明取得了最佳平衡。其中 XGBoost 与 BP Neural Network 的 Macro AUC = 1.000（完美分类），其次为 SVM，Macro AUC = 0.998（接近完美）。并且五种模型均取得了极优的性能（AUC > 0.99），说明轴承故障诊断任务在当前特征工程下是高度可解的。

（三）基于对五种不同模型的比较与评估，本文分别绘制了五个混淆矩阵，详细揭示了五种模型具体在哪些类别上犯了错，以及犯了什么类型的错误，如图 13所示。

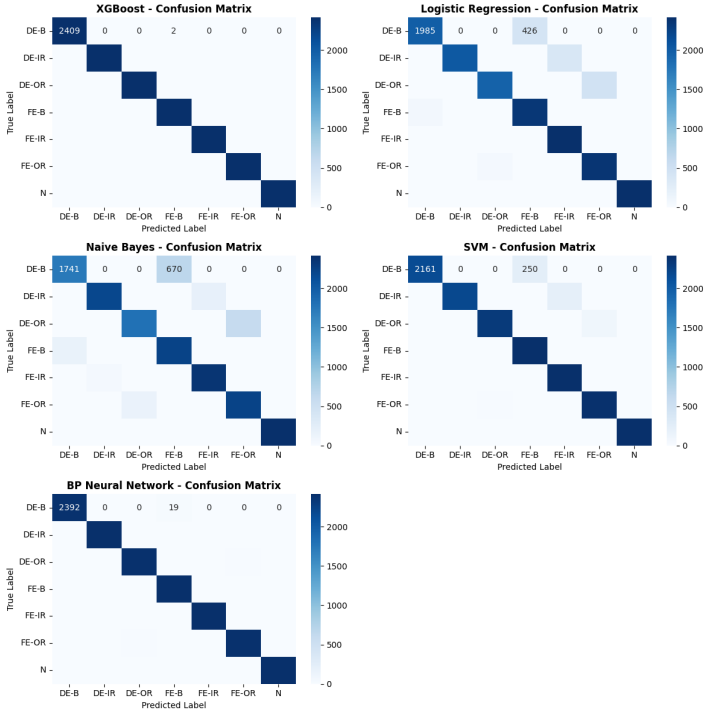


图 13　五种模型的混淆矩阵图

上图展示了五种不同模型整体准确度以及错误类别，本文发现只有 XGBoost 的所有样本都集中在对角线上，而其他模型在非对角线上的单元格均存在样本，如 BP Neural Network 中 DE-B 被误判为 FE-B 的样本存在 19 个，而 Naive Bayes 中 DE-B 被误判为 FE-B 的样本却存在了 670 个，这说明了在这五种模型中，XGBoost 模型整体表现最优，而 Naive Bayes 的整体表现最差。

5.4 问题二小结与分析

针对问题二，本文通过过采样方法平衡数据集后，按照 7：3 的比例划分了源域数据集为训练集与测试集。并选用了五个模型进行训练，按照准确率进行模型排名，第一名为 XGBoost，其准确率为 0.9991；第二名为 BP Neural Network，其准确率为 0.9972；第三名为 SVM，其准确率为 0.9655；第四名为 Logistic Regression，其准确率为 0.9167；最后一名为 Naive Bayes，其准确率为 0.8889。因此最佳诊断模型为 XGBoost，进一步优化可考虑超参数调优。

6 问题三的求解

6.1 问题三分析

本问题的核心工作为两个关键步骤：其一，考虑到源域与目标域数据共性与差异性特征，基于问题二中的诊断模型，设计合适的迁移学习方法，建立新的诊断模型，用于对目标域中未知标签的数据样本进行分类和标定。其二，将迁移结果可视化和分析，给出目标域样本的分类和标定结果。

针对源域与目标域的分布偏移问题（差异性特征），本文先通过特征空间的统一化处理，对目标域实施与第二问相同的特征提取和主成分分析降维处理（聚焦共性特征）。在此基础上，构建有约束的迁移学习-ABC 算法和无约束迁移学习-ABC 算法，对目标域样本进行分类与标定。

针对结果可视化问题，本文采用综合可视化方案（域差异变化、约束违反程度、置信度分布等）全面展示迁移学习过程的收敛性和结果的可靠性。

问题三的解题步骤流程如图 3 所示。

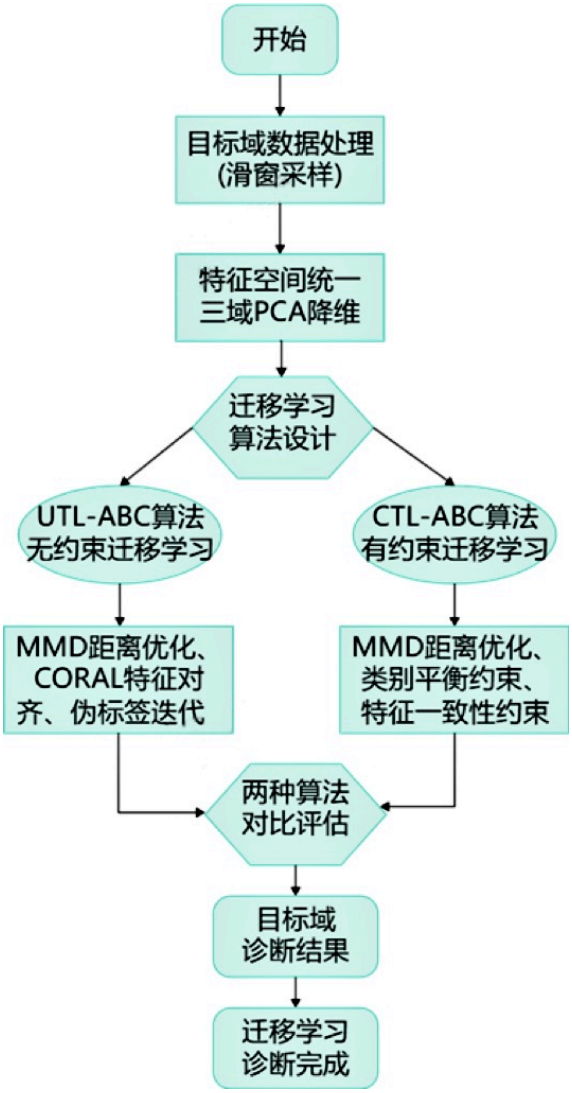


图 14　问题三技术路线图

6.2 目标域数据集处理

6.2.1 源域与目标域数据处理方法对比

由材料可知，目标域仅有 16 个待检测和标定的样本数据，数据量较小。而大多数机器学习模型，需要足够的数据量才能有效的提取数据特征。因此本文基于轴承故障机理，结合源域和目标域的故障类别约束与故障特征约束，通过窗口滑动采样方法，设置窗口大小为 1 万个数据点，步长为 1000 个数据点，将目标域样本个数从 16 个扩充到了 3952 个，便于后续数据集的特征提取。为统一源域与目标域数据的特征空间，本文继续施用问题二中的三域（时域、频域、时频域）特征提取方案，选取 StandardScaler 方法进行数据处理，并用 PCA 降维至 13 维，为后续 XGBoost 模型诊断目标域做准备。源域与目标域数据预处理的方法如表 5所示。

表 5　源域与目标域数据预处理方法对比

处理环节	源域	目标域	一致性
滑动窗口采样	窗口大小：由 T_i 决定 步长：故障周期决定	窗口大小：10000 步长：1000	不一致
多域特征提取	时域、频域、时频域	时域、频域、时频域	一致
数据标准化	StandardScaler 方法	StandardScaler 方法	一致
数据平衡处理	SMOTE/过采样	无（标签未知）	不一致
故障标签处理	7 种故障标签	无（标签未知）	不一致
输出维度	13	13	一致

6.2.2 基于目标域的 PCA 特征降维

本文对降维的结果进行了可视化分析，如图 15所示。

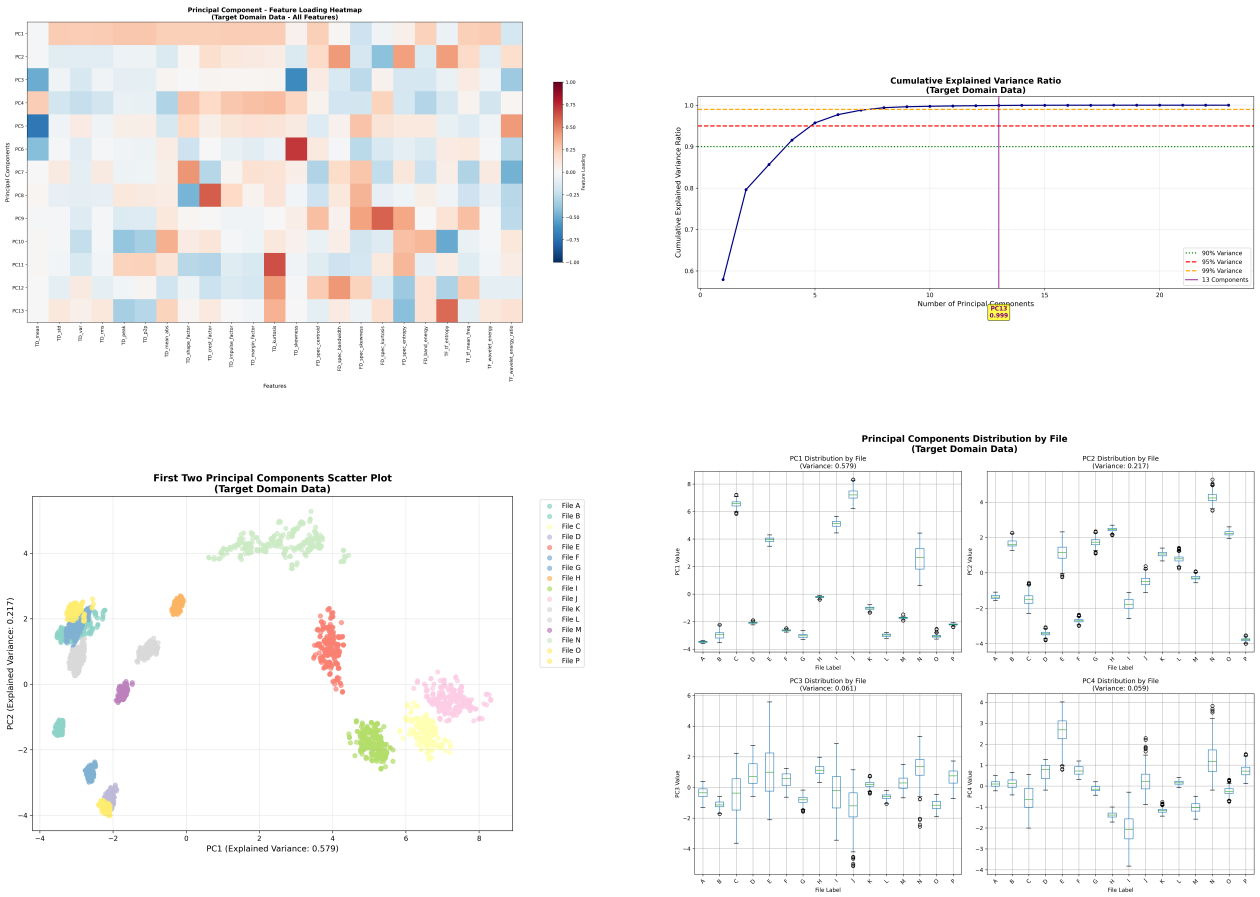


图 15　基于目标域的 PCA 降维可视化分析

由上图可知，本文可以得出 13 个主成分的累计方差解释率占比为 99.9%，并且每个类别形成紧密的簇群，簇间有清晰的边界，说明特征提取有效，故障模式可划分，并且迁移学习可行性高。

同时，本文绘制了一个目标域上的三维主成分分析（PCA）散点图，展示了前三个主成分的方差解释率及目标域数据的分布情况，可视化如图 16所示。

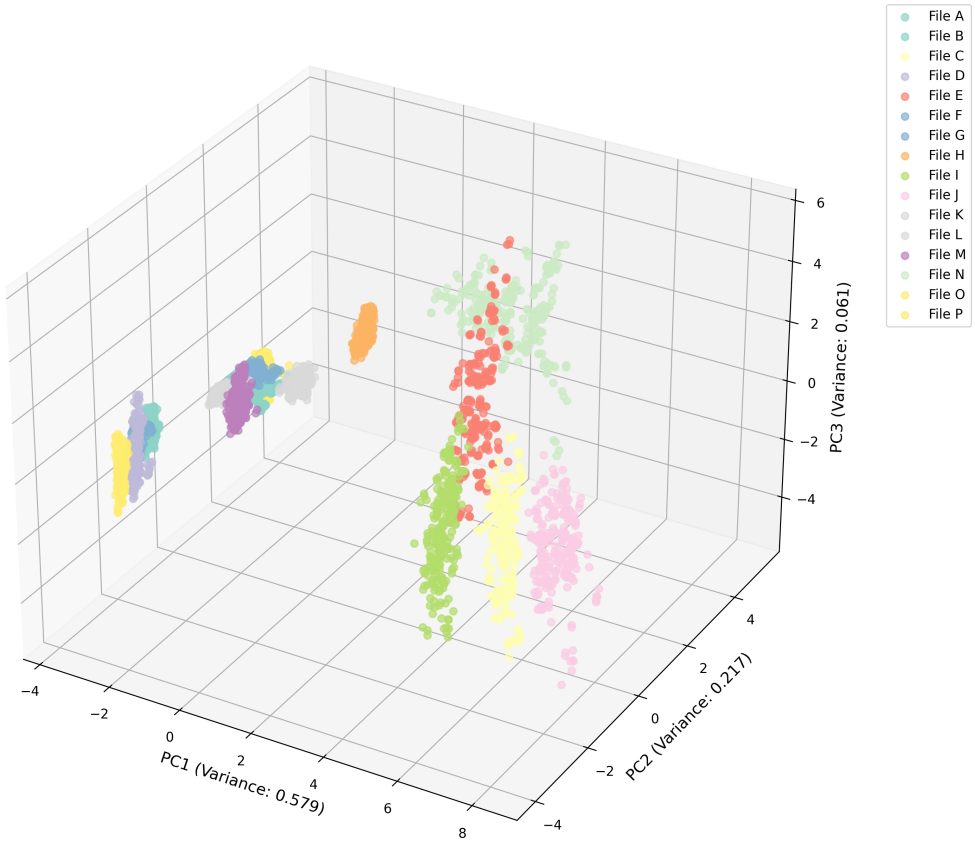


图 16　前三个主成分的 3D 散点图

由上图可知，PC 1 的方差解释率达到了 0.579（即 57.9%），说明第一个主成分包含了数据中大部分信息；其次为 PC 3，其方差解释率为 0.317（即 31.7%），是第二重要的成分；PC 1 和 PC 3 共同解释了几乎 89.6% 的总方差，是数据变异的主要来源。而 PC 2 的方差解释率为 0.061（6.1%），贡献较小。

6.3 UTL-ABC 算法

6.3.1 UTL-ABC 算法原理

在处理后的目标域数据集上，本文以 XGBoost 模型为基础，结合迭代优化策略实现对模型的渐进域适应，建立 XGBoost+UTL-ABC 的无约束迁移诊断模型。模型采用置信度引导的伪标签迭代优化方法，以 XGBoost 模型生成目标域初始伪标签，通过动态置信度阈值筛选高可靠性的样本参与训练。在优化过程中，结合 CORAL 特征对齐减少域间差异，并利用 MMD 度量监控收敛状态。基于 XGboost 的 UTL-ABC 算法的计算公式如下：

1. **最大均值差异 (MMD)**

$$MMD^2(X_s, X_t) = \left\| \frac{1}{n_s} \sum_{i=1}^{n_s} \phi(x_i^s) - \frac{1}{n_t} \sum_{j=1}^{n_t} \phi(x_j^t) \right\|_{\mathcal{H}}^2 \tag{6.28}$$

使用 RBF 核函数展开：

$$MMD = \sqrt{\frac{1}{n_s^2} \sum_{i,j} K(x_i^s, x_j^s) + \frac{1}{n_t^2} \sum_{i,j} K(x_i^t, x_j^t) - \frac{2}{n_s n_t} \sum_{i,j} K(x_i^s, x_j^t)} \tag{6.29}$$

其中，X_s 和 X_t：分别表示源域数据和目标域数据，n_s 和 n_t：分别表示 X_s 和 X_t 中的样本数量，x_i^s 表示 X_s 中的第 i 个样本，x_j^t 同理。

2. **CORAL 特征对齐**

协方差对齐公式：

$$\min_A \|C_s - AC_tA^T\|_F^2 \tag{6.30}$$

其中，$C_s = \frac{1}{n_s-1}X_s^TX_s$ 为源域协方差矩阵，$C_t = \frac{1}{n_t-1}X_t^TX_t$ 为目标域协方差矩阵。

变换公式：

$$X_s^{aligned} = (X_s - \mu_s)C_s^{-1/2}C_t^{1/2} + \mu_t \tag{6.31}$$

其中，$X_s^{aligned}$ 表示对齐后的源域特征，μ_s 表示源域特征的均值变量，C_s 表示源域特征的协方差矩阵，C_t，μ_t 同理，

3. **伪标签置信度**

$$conf_i = \max_k P(y_i = k|x_i^t;\theta) \tag{6.32}$$

其中，$conf_i$ 表示第 i 个样本的伪标签置信度，y_i 表示第 i 个样本的真实标签，k 为标签类别，x_i^t 表示目标域中的第 i 个样本，θ 为模型参数。

4. **简化的优化目标（无约束项）**

$$\min_\theta \mathcal{L}_{total} = \mathcal{L}_{cls} + \lambda_1 \cdot MMD + \lambda_2 \cdot \mathcal{L}_{pseudo} + \lambda_3 \cdot \|\theta\|_2 \tag{6.33}$$

其中，$\mathcal{L}_{total}$ 表示总损失函数，$\mathcal{L}_{cls}$ 表示分类损失，$\|\theta\|_2$ 表示 L_2 正则化项。

5. **样本权重**

$$w_i = \begin{cases} 1.0, & \text{if } x_i \in \mathcal{D}_s \\ conf_i, & \text{if } x_i \in \mathcal{D}_t^{high} \end{cases} \tag{6.34}$$

其中，$\mathcal{D}_t^{high}$ 表示目标域高置信度样本集，$\mathcal{D}_s$ 同理。

基于 XGboost 的 UTL-ABC 的步骤如算法 1 所示。

算法 1 UTL-ABC 无约束迁移学习算法

输入: 源域数据 (X_s, Y_s)；目标域数据 X_t；预训练模型 f_θ；参数 $\lambda_1, \lambda_2, \lambda_3$；最大迭代次数 T；置信度阈值 τ。

输出: 目标域预测标签 $\hat{Y}_t$ 及对应置信度 $conf_t$。

function UTL-ABC($(X_s, Y_s), X_t, f_\theta, \lambda_1, \lambda_2, \lambda_3, T, \tau$)

    步骤 1: 初始化

    加载预训练的 XGBoost 分类器 f_θ

    计算源域类别分布 p_s（仅用于监控，不作为约束）

    初始化对齐特征 $X_s^{(0)} \leftarrow X_s, X_t^{(0)} \leftarrow X_t$

    设置动态阈值 $\tau^{(0)} \leftarrow 0.7$

    设目标域样本数为 n_t

    初始化前一轮预测标签 $\hat{Y}_t^{(-1)} \leftarrow \emptyset$

    步骤 2: 迭代优化

    for $t \leftarrow 1$ **to** T **do**

        // 计算域差异

        $d_{\text{MMD}} \leftarrow \text{MMD}(X_s^{(t-1)}, X_t^{(t-1)})$

        // 特征对齐

        if $t > 1$ **then**

            $(X_s^{(t)}, X_t^{(t)}) \leftarrow \text{CORAL}(X_s, X_t)$

        else

            $X_s^{(t)} \leftarrow X_s^{(t-1)}, X_t^{(t)} \leftarrow X_t^{(t-1)}$

        end if

        // 伪标签生成

        $P_t \leftarrow f_\theta(X_t^{(t)})$ ▷ 概率预测

        $\hat{Y}_t^{(t)} \leftarrow \arg\max(P_t)$ ▷ 硬标签

        $conf_t^{(t)} \leftarrow \max(P_t)$ ▷ 置信度

        // 监控分布（无约束强制）

        计算目标域预测分布 p_t ▷ 输出用于观察，但不进行约束调整

        // 筛选高置信样本

        $\mathcal{D}_{\text{high}} \leftarrow \{(x_i^t, \hat{y}_i^t) \mid conf_i^{(t)} > \tau^{(t)}\}$

        $n_{\text{high}} \leftarrow |\mathcal{D}_{\text{high}}|$

算法 1 UTL-ABC 无约束迁移学习算法（续）

// 模型更新
if $n_{\text{high}} > 10$ **then**
采样源域: $\mathcal{D}_s^{\text{sample}} \sim \mathcal{D}_s$ ▷ size = min(2000, n_s)
构建训练集: $\mathcal{D}_{\text{train}} \leftarrow \mathcal{D}_s^{\text{sample}} \cup \mathcal{D}_{\text{high}}$
// 权重分配
for $x_i \in \mathcal{D}_{\text{train}}$ **do**
if $x_i \in \mathcal{D}_s^{\text{sample}}$ **then**
$w_i \leftarrow 1.0$
else
$w_i \leftarrow conf_i^{(t)}$ ▷ 目标域样本权重为置信度
end if
end for
更新模型: $f_\theta \leftarrow \text{train}(f_\theta, \mathcal{D}_{\text{train}}, w)$
end if
// 计算损失
$\mathcal{L}^{(t)} \leftarrow \lambda_1 \cdot d_{\text{MMD}}$
// 收敛判断
if $t > 1$ **then**
$r_{\text{change}} \leftarrow \|\hat{Y}_t^{(t)} - \hat{Y}_t^{(t-1)}\|_0 / n_t$
if $r_{\text{change}} < 0.01$ **then**
break
end if
end if
// 自适应阈值调整
if $t > 3$ **and** $n_{\text{high}} < 0.3 \cdot n_t$ **then**
$\tau^{(t+1)} \leftarrow \max(0.5, \tau^{(t)} - 0.05)$
else
$\tau^{(t+1)} \leftarrow \tau^{(t)}$
end if
$\hat{Y}_t^{(t-1)} \leftarrow \hat{Y}_t^{(t)}$ ▷ 更新历史标签
end for
步骤 3: 输出最终预测
对每个文件进行众数投票聚合
计算文件级平均置信度
return $(\hat{Y}_t, conf_t)$
end function

6.3.2 UTL-ABC 参数设置

为确保算法的有效性和可重复性，本文对 UTL-ABC 无约束迁移学习算法的关键参数进行了详细设置，具体参数配置如表 6 所示。

表 6　UTL-ABC 算法部分训练参数设置

参数	含义	取值
T	最大迭代次数	50
ϵ	类别分布差异的容忍阈值	0.15
δ	特征中心距离阈值	1.0
λ_1	MMD 损失权重	0.3
λ_2	伪标签损失权重	0.5
λ_3	正则化权重	0.1
τ	置信度阈值	0.7

6.3.3 基于 UTL-ABC 结果的可视化分析

根据上面的参数训练，结果生成了基于 UTL-ABC 算法 (自适应平衡控制的无约束迁移学习) 训练过程的多子图可视化结果，如图 17所示。

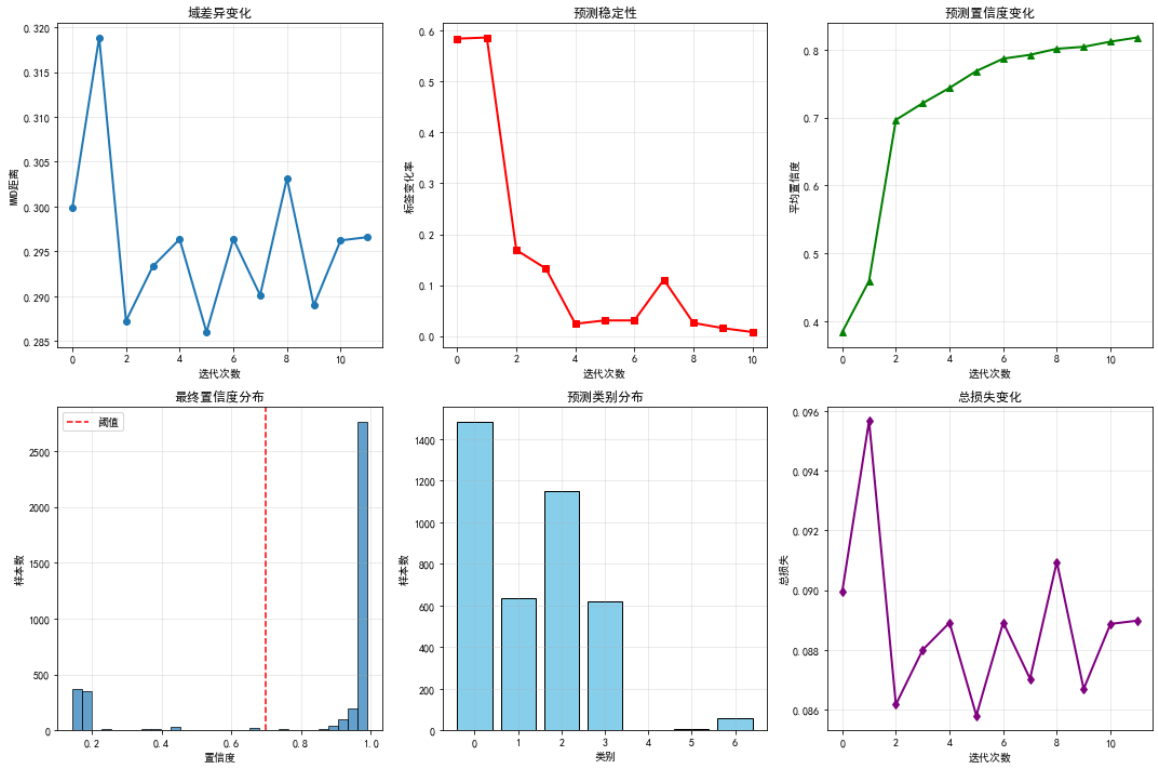


图 17　UTL-ABC 训练结果图

由上图可知，基于 XGBoost 的 UTL-ABC 无约束迁移学习算法的训练过程动态变化。从上面六个图的结合分析能够发现模型的收敛过程以及迁移效果：

（1）MMD 距离（左上）由 0.3 通过迭代一次上升到 0.318，然后急剧下降并在 0.285 与 0.305 之间来回波动，最终在 0.295 附近稳定，说明源域和目标域之间的分布差异在迭代过程中得到有效的缩减；

（2）标签变化率（中上）由 0.58 急剧下降至 0.04 以下，在第 10 次迭代时收敛到零，证明了算法的快速收敛特性；

（3）平均置信度（右上）由 0.38 持续提升至 0.82，提升幅度达 115.8%，对于整体而言平均的置信度在提升，从而也说明的模型对目标域样本的诊断可靠性在明显增强。整体而言，该训练过程展现了良好的收敛性和稳定性，验证了 UTL-ABC 算法在跨域故障诊断任务中的有效

(4) 最终置信度分布（下）图中处在置信度高的区域的样本数量居多，约有 2750 条样本，约 800 条样本处在置信度低的区域，这充分有效的说明了模型的有效性和可靠性；

(5) 预测类别分布（下）显示了 UTL-ABC 算法对目标域的类别预测的分布情况，其集中趋势表明模型学习到了有效的故障特征表

(6) 总损失函数由 0.9 以波动的形式下降，最终稳定在 0.88 到 0.885 之间，表明模型优化过程稳定有效。

最终基于 XGBoost 的 UTL-ABC 无约束迁移学习算法输出的目标域的预测结果如下表：

表 7　UTL-ABC 目标域轴承故障诊断的预测结果

文件	预测标签	置信度	可靠性	文件	预测标签	置信度	可靠性
A	DE-B	0.479	低	I	FE-B	0.927	高
B	DE-B	0.987	高	J	DE-B	0.824	高
C	DE-B	0.857	高	K	FE-B	0.329	低
D	DE-OR	0.952	高	L	DE-B	0.624	中
E	DE-IR	0.978	高	M	FE-B	0.517	低
F	DE-OR	0.909	高	N	DE-IR	0.957	高
G	DE-B	0.842	高	O	DE-B	0.979	高
H	DE-OR	0.979	高	P	DE-OR	0.947	高

如表 7 可知，UTL-ABC 已完成对目标域 16 个样本的故障诊断分类，根据分类的结果预测可知 DE-B 的占比最高，高达 43.75%，其次是 DE-OR 占比 25%，DE-IR 占比 12.5%，以及 FE-B 占比 18.75%；同时，在置信度方面，置信度大于 0.8 的样本预测可靠性高，置信度介于 0.6-0.8 之间的样本预测可靠性中，置信度低于 0.6 的样本预测可靠性低；在可靠性方面，高可靠性的样本的数量为 12 个，中可靠性的样本数量为 1 个，低可靠的样本数量为 3 个。整体而言，该算法展示了较好的诊断能力。

6.4 CTL-ABC 算法

6.4.1 CTL-ABC 算法原理

与无约束迁移诊断模型的算法类似，本文在无约束算法基础上，增加类别平衡约束和特征一致性约束，建立 XGBoost+CTL-ABC 的有约束迁移诊断模型。基于 XGboost 的 CTL-ABC 算法计算公式（补充约束部分和总体优化目标）如下：

1. **类别平衡约束**

$$|P(Y_t = k) - P(Y_s = k)| \leq \epsilon, \quad \forall k \in \{1, 2, ..., K\} \tag{6.35}$$

违反程度：

$$V_{balance} = \sum_{k=1}^{K} \max(0, |p_k^t - p_k^s| - \epsilon)^2 \tag{6.36}$$

其中，$P\left(Y_s = k\right)/p_k^s$ 表示源域中类别 k 的先验概率/样本比例，$P\left(Y_t = k\right)/p_k^t$ 同理。

2. **特征一致性约束**

$$\|\mu_k^s - \mu_k^t\|_2 \leq \delta, \quad \forall k \tag{6.37}$$

其中：$\mu_k^s = \frac{1}{n_k^s} \sum_{x_i \in C_k^s} x_i$ 为源域类 k 的中心，$\mu_k^t = \frac{1}{n_k^t} \sum_{x_j \in C_k^t} x_j$ 为目标域类 k 的中心。

3. **总体优化目标**

$$\min_{\theta} \mathcal{L}_{total} = \mathcal{L}_{cls} + \lambda_1 \cdot MMD + \lambda_2 \cdot \mathcal{L}_{pseudo} + \alpha_1 \cdot V_{balance} + \alpha_2 \cdot V_{consistency} \tag{6.38}$$

其中：$\mathcal{L}_{\text{total}}$ 为总损失，$\mathcal{L}_{\text{cls}}$ 是分类损失，$\mathcal{L}_{\text{pseudo}}$ 为伪标签损失，V_{balance} 是类别平衡损失，$V_{\text{consistency}}$ 是类别一致性损失，θ 为模型参数。

基于 XGboost 的 CTL-ABC 算法 2 步骤如下所示。

算法 2 CTL-ABC 迁移学习算法

输入： 源域数据 (X_s, Y_s)；目标域数据 X_t；预训练模型 f_θ；参数 $\epsilon, \delta, \lambda_1, \lambda_2, \alpha_1, \alpha_2$；最大迭代次数 T；置信度阈值 τ。

输出： 目标域预测标签 $\hat{Y}_t$ 及对应置信度 $conf_t$。

function CTL-ABC($(X_s, Y_s), X_t, f_\theta, \epsilon, \delta, \lambda_1, \lambda_2, \alpha_1, \alpha_2, T, \tau$)

步骤 1: 初始化

加载预训练的 XGBoost 分类器 f_θ

计算源域类别分布 $p_s = [p_1^s, p_2^s, \ldots, p_K^s]$

初始化对齐后的特征 $X_s^{(0)} \leftarrow X_s$

$X_t^{(0)} \leftarrow X_t$

初始化前一轮预测标签 $\hat{Y}_t^{(-1)} \leftarrow \emptyset$

设目标域样本数为 n_t

步骤 2: 迭代优化

for $t \leftarrow 1$ **to** T **do**

// 域适应：计算 MMD 距离

$d_{\text{MMD}} \leftarrow \text{MMD}(X_s^{(t-1)}, X_t^{(t-1)})$

// 特征对齐：CORAL 对齐

$(X_s^{(t)}, X_t^{(t)}) \leftarrow \text{CORAL}(X_s^{(t-1)}, X_t^{(t-1)})$

// 伪标签生成

$P_t \leftarrow f_\theta(X_t^{(t)})$

$\hat{Y}_t^{(t)} \leftarrow \arg\max(P_t)$

$conf_t^{(t)} \leftarrow \max(P_t)$

// 约束检查

计算目标域当前类别分布 $p_t = [p_1^t, \ldots, p_K^t]$

$V_{\text{balance}} \leftarrow \sum_k \max(0, \|p_k^t - p_k^s\| - \epsilon)^2$

$V_{\text{consistency}} \leftarrow \sum_k \max(0, \|\mu_k^s - \mu_k^t\| - \delta)^2$

// 模型更新

选择高置信度样本: $\mathcal{D}_{\text{high}} = \{(x_i^t, \hat{y}_i^t) \mid conf_i^{(t)} > \tau\}$

构建增强训练集: $\mathcal{D}_{\text{train}} = \mathcal{D}_s \cup \mathcal{D}_{\text{high}}$

// 样本权重调整

for $x_i \in \mathcal{D}_{\text{train}}$ **do**

if $x_i \in \mathcal{D}_s$ **then**

$w_i \leftarrow 1.0$

else

$w_i \leftarrow conf_i^{(t)} \cdot \beta(V)$

end if

end for

算法 2 CTL-ABC 迁移学习算法（续）

更新模型: $f_\theta \leftarrow \text{train}(f_\theta, \mathcal{D}_{\text{train}}, w)$

// 收敛判断

if $\frac{\|\hat{Y}_t^{(t)} - \hat{Y}_t^{(t-1)}\|_0}{n_t} < 0.01$ **and** $V_{\text{balance}} < 0.1$ **then**

break

end if

$\hat{Y}_t^{(-1)} \leftarrow \hat{Y}_t^{(t)}$

end for

步骤 3: 输出最终预测

对每个文件聚合预测结果（众数投票）

计算每个文件的平均置信度

return $(\hat{Y}_t, conf_t)$

end function

6.4.2 CTL-ABC 参数设置

下面是本文对 CTL-ABC 有约束迁移学习算法的关键参数进行了详细设置，具体参数配置如表 8 所示。

表 8　关键参数说明

参数	含义	取值
T	最大迭代次数	50
ϵ	类别分布差异的容忍阈值	0.15
δ	特征中心距离阈值	1.0
λ_1	MMD 损失权重	0.3
λ_2	伪标签损失权重	0.5
λ_3	正则化权重	0.5
α_1	类别平衡约束权重	1.0
α_2	特征一致性约束权重	0.5
τ	置信度阈值	0.7

6.4.3 基于 CTL-ABC 结果的可视化分析

同时，根据上面的参数表训练结果绘制了一个基于 CTL(自适应平衡控制的无约束迁移学习) 模型训练过程的多子图可视化结果，如图 18所示。

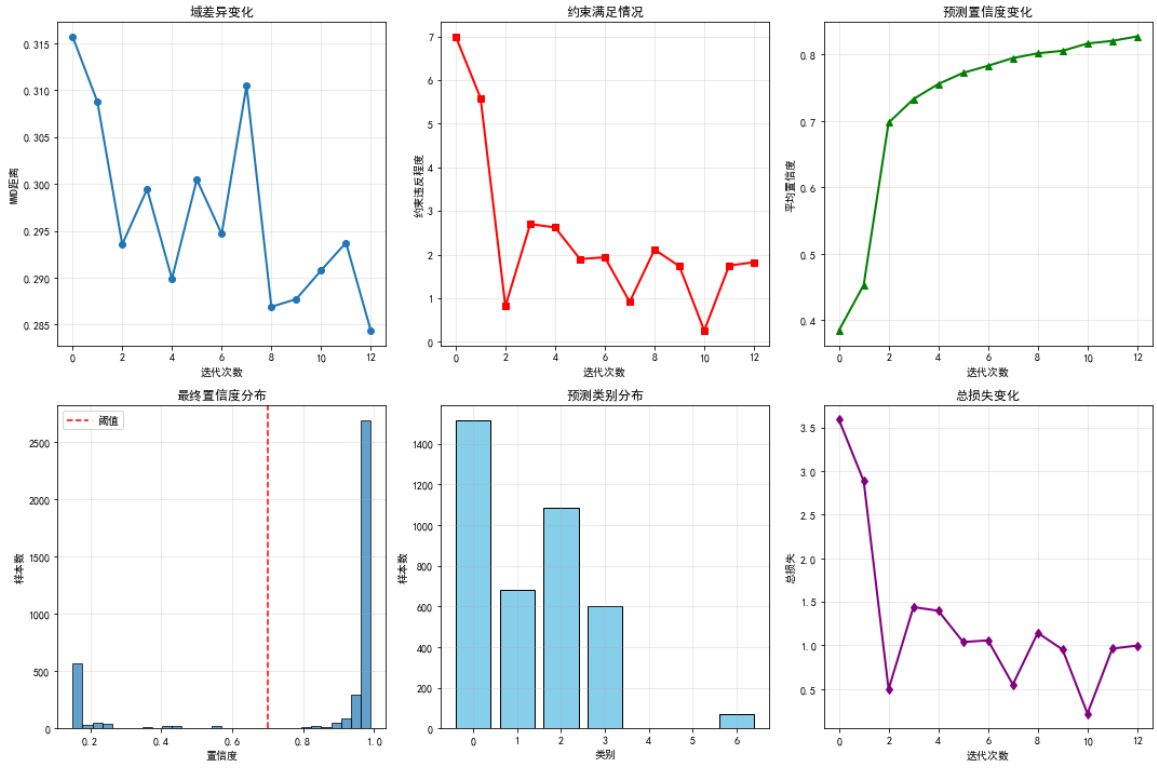


图 18　CTL-ABC 训练结果图

上图展示了 CTL-ABC 有约束迁移学习算法的训练表现优异：其中 MMD 距离从 0.317 降至 0.29 左右并稳定，源域与目标域分布差异有效缩减；约束违反程度前 2 次迭代从 7.0 剧降至近 0.9，最终平稳在 1.8 左右，较好平衡类别分布与数据特征一致性；预测置信度随迭代稳步上升至约 0.83，大量样本（约 2700 个）集中在 0.8-1.0 高置信度区间，模型可靠性显著提升；预测类别分布体现模型学到有效故障特征分布规律；总损失函数从 3.7 下降后在 0.5-1.5 波动并趋于稳定，整体训练的收敛性和稳定性好，验证了该算法引入类别平衡和特征一致性约束，能有效实现轴承故障诊断知识迁移。

同时基于 XGBoost 的 CTL-ABC 有约束迁移学习算法输出的目标域的预测结果如下表：

表 9　CTL-ABC 目标域轴承故障诊断的预测结果

文件	预测标签	置信度	可靠性	文件	预测标签	置信度	可靠性
A	DE-B	0.546	低	I	FE-B	0.911	高
B	DE-B	0.986	高	J	DE-B	0.824	高
C	DE-B	0.853	高	K	FE-B	0.288	低
D	DE-OR	0.965	高	L	DE-B	0.721	中
E	DE-IR	0.978	高	M	FE-B	0.484	低
F	DE-OR	0.926	高	N	DE-IR	0.957	高
G	DE-B	0.871	高	O	DE-B	0.980	高
H	DE-OR	0.977	高	P	DE-OR	0.966	高

如表 9 可知，CTL-ABC 已完成对目标域 16 个样本的故障诊断分类，根据分类的结果预测可知 DE-B 的占比最高，高达 43.75%，其次是 DE-OR 占比 25%，DE-IR 占比 12.5%，以及 FE-B 占比 18.75%；该预测的结果和 UTL-ABC 的预测结果一样，说明该算法也与 UTL-ABC 算法拥有较好的诊断能力。

6.5 基于两种算法的评估实验

本文对 CTL - ABC 和 UTL - ABC 两种迁移学习算法的总损失变化、预测置信度、训练时间等多维度进行了对比分析，如图 19所示。

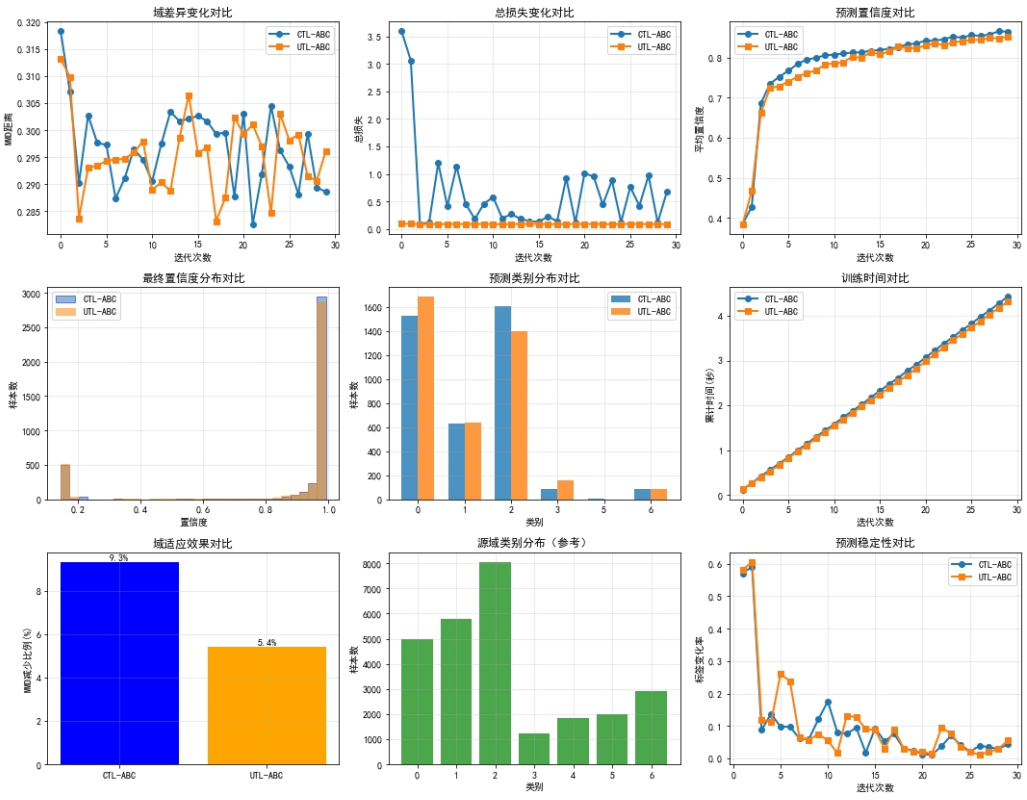


图 19　两种迁移学习算法对比分析图

由上图可知，CTL-ABC 在总损失控制、置信度、域差异处理、域适应效果以及预测稳定性等综合性能指标上，整体优于 UTL-ABC；而 UTL-ABC 则在训练时间效率上表现更好。本文更看重算法的迁移学习效果与预测可靠性，所以 CTL-ABC 是更优选择。

基于两种算法（CTL-ABC 与 UTL-ABC）的诊断预测，算法预测一致性可视化分析如图 20所示。

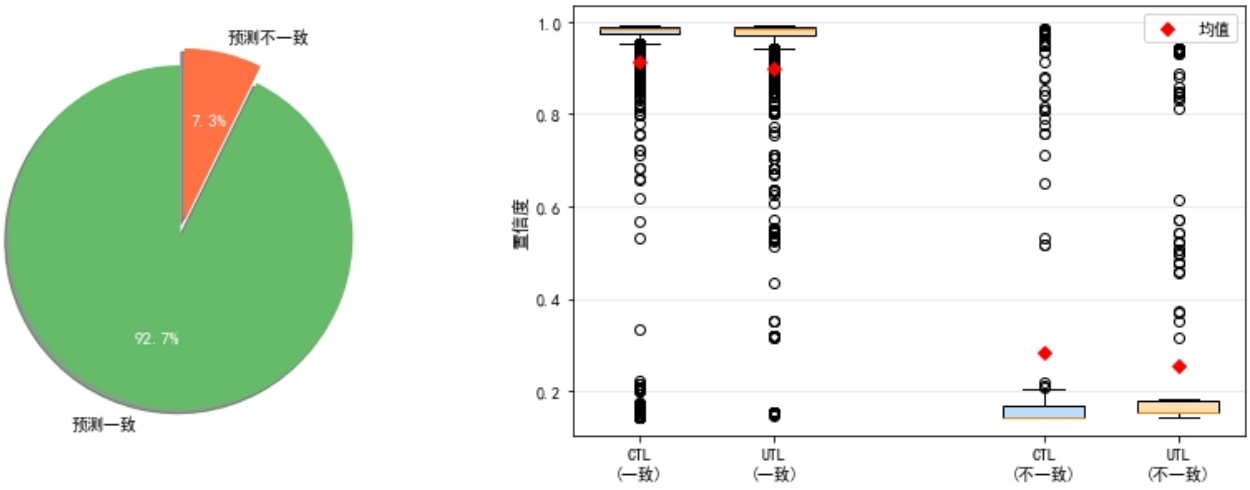


图 20　算法预测一致性分析

从饼图可知，整体预测一致的比例高达 92.7%，而 CTL-ABC 在预测一致时，与 UTL-ABC 一样有着较高的置信度。而在预测不一致的情况下，CTL-ABC 的置信度均值明显低于 UTL-ABC。这说明了当 CTL-ABC 算法出现预测不一致时，它对自身预测结果的“自信程度”较低。相比之下，UTL-ABC 在预测不一致时仍保持较高置信度，可能会让使用者误判预测的可靠性。

结合 CTL-ABC 约束违反程度随迭代次数的变化, 绘制了 CTL-ABC 算法相应的约束违反程度图如图 21所示。

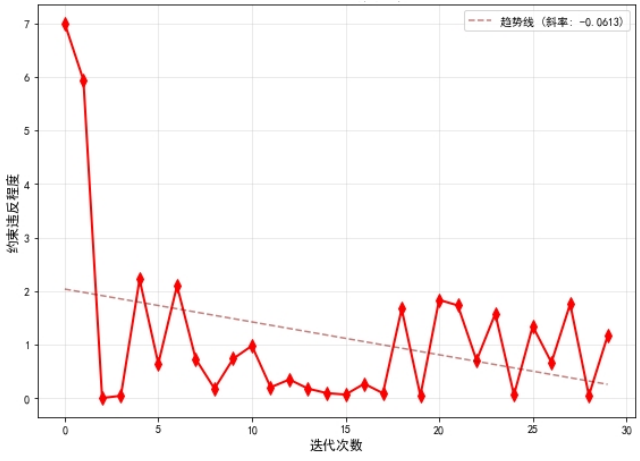


图 21　CTL-ABC 约束满足情况

由上图可知，约束违反程度的趋势线斜率为 -0.0613，整体呈下降趋势，并且整体约束违反程度从初始较高值快速降低后，虽有波动但整体处于相对较低的水平并逐渐趋于平稳。因此 CTL-ABC 在约束处理方面表现出色，能更高效地降低约束违反程度，这有助于诊断模型更好地平衡各类分布及保持数据特征的一致性。

6.6 问题三小结与分析

基于两种算法的性能综合对比总结如表 10所示。(其中 CTL-ABC 算法的优势项为 6，UTL-ABC 算法的优势项为 1)

表 10　算法性能综合对比总结

指标	CTL-ABC	UTL-ABC	优势算法
最终 MMD	0.2886	0.2962	CTL-ABC
平均置信度	0.865	0.853	CTL-ABC
最高置信度	0.992	0.991	CTL-ABC
置信度标准差	0.281	0.290	CTL-ABC
迭代次数	30	30	UTL-ABC
MMD 减少率 (%)	9.32	5.43	CTL-ABC
最终约束违反	1.1667	N/A	CTL-ABC

从上表可得:（一）CTL-ABC 最终 MMD 为 0.2886，低于 UTL-ABC 的 0.2962，说明 CTL-ABC 更能更高效地缩减源域与目标域的分布差异；（二）CTL-ABC 的平均置信度 0.865，最高置信度为 0.992，二者均高于 UTL-ABC，置信度标准差 0.281 低于 UTL-ABC 的 0.29，显示 CTL-ABC 的预测结果不仅可信度更高，且置信度的稳定性更好，预测更可靠；（三）CTL-ABC 存在明确的最终约束违反值 1.1667，而 UTL-ABC 无有效数据，体现出 CTL-ABC 在约束处理方面表现更优秀，并且能更好地平衡各类分布与保持数据特征的一致性。

小结与分析：CTL-ABC 和 UTL-ABC 算法性能的综合对比，虽然迭代次数上两个算法均为 30 次，UTL-ABC 无明显优势，但综合考虑其他核心指标来看，CTL-ABC 在诊断迁移学习的整体性能表现上展现出了显著优势，因此本文最终选用了 CTL-ABC 算法（自适应平衡控制的约束迁移学习）作为目标域的诊断模型。

7 问题四的求解

7.1 问题四分析

由于机器学习模型的“黑箱”问题，其迁移和诊断过程难以被观测和理解，这可能造成使用者对模型结果的不信任或盲目信任，进而影响诊断模型的应用。因此，本问题的核心工作为结合轴承故障的特点和机理，综合考虑第三问中模型的结构设计、迁移和决策过程，对迁移诊断的可解释性进行分析。

针对问题四，本文基于第三问的故障检测模型，选择迁移诊断的事前/迁移过程/事后可解释性三方面进行详细的分析。

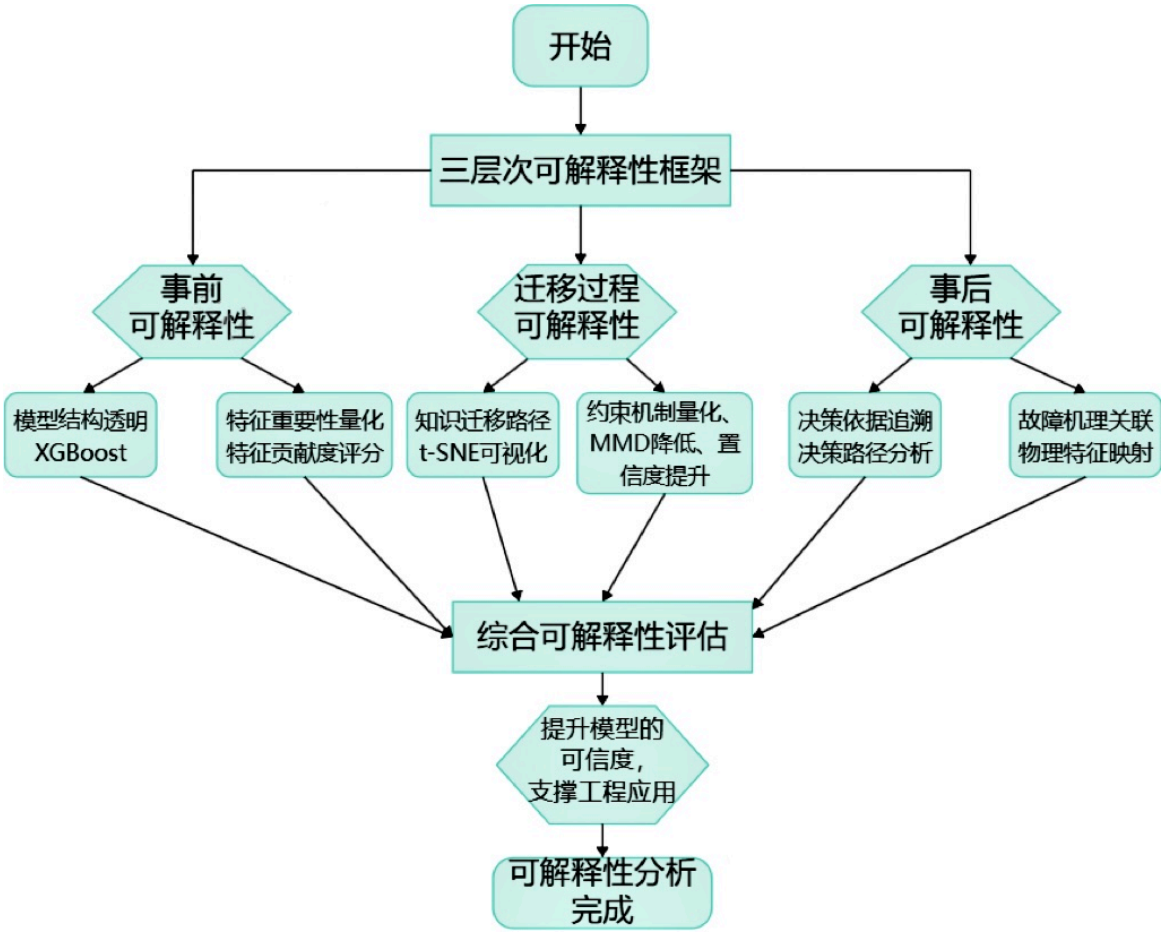


图 22　问题四技术路线图

7.2 事前可解释性分析

7.2.1 模型结构的透明性

事前可解释性关注模型本身的透明结构和决策逻辑。本文采用 XGBoost 作为基础诊断模型，该模型具有天然的可解释性优势，具体算法公式详见本文第二问中的“XGBoost”部分。

因为 XGBoost 的决策过程可表示为多个决策树的加法模型，其每个分裂节点都对应明确的特征阈值规则，其决策路径是可追溯的。此外，XGBoost 对特征重要性的评分，可以直观的反映各特征对模型故障诊断的贡献度，为模型提供了透明化的解释依据。

7.2.2 特征重要性量化

为增强模型的可解释性，本文计算了各特征对故障诊断的贡献度。特征 j 的重要性得分定义为：

$$I_j = \frac{1}{K}\sum_{k=1}^{K}\sum_{t\in T_k} \mathbb{1}[v(t)=j]\cdot G_t \tag{7.39}$$

其中，T_k 为第 k 棵树的所有非叶节点集合，$v(t)$ 为节点 t 使用的特征索引，G_t 为节点 t 的增益：

$$G_t = \frac{1}{2}\left[\frac{G_L^2}{H_L+\lambda}+\frac{G_R^2}{H_R+\lambda}-\frac{(G_L+G_R)^2}{H_L+H_R+\lambda}\right]-\gamma \tag{7.40}$$

其中，G_L、G_R 分别为左右子节点的一阶梯度和，H_L、H_R 为二阶梯度和，λ 和 γ 为正则化参数。

7.3 迁移过程可解释性分析

7.3.1 知识迁移路径可视化

为揭示知识从源域到目标域的迁移路径，本文采用 t-SNE 降维技术将高维特征空间映射到二维平面进行可视化分析。

t-SNE 的目标函数为：

$$C = \sum_i \sum_j p_{ij}\log\frac{p_{ij}}{q_{ij}} \tag{7.41}$$

其中，p_{ij} 为高维空间中样本 i 和 j 的联合概率，q_{ij} 为低维空间中的对应概率。

如图 23所示，左图展示了源域数据在二维空间的分布，各故障类型形成了明显的聚类簇，表明特征提取是有效的。右图展示了目标域数据经过迁移学习后的预测分布，可以观察到目标域样本在特征空间中的分布与源域相应故障类别的分布具有一定的相似性，证明了两个数据域存在相同的故障特征。且不同故障类型（DE-B、DE-IR、DE-OR 等）在目标域中均有良好的可区分性。驱动端故障与风扇端故障呈现的不同分布方式，符合轴承故障的物理机理。

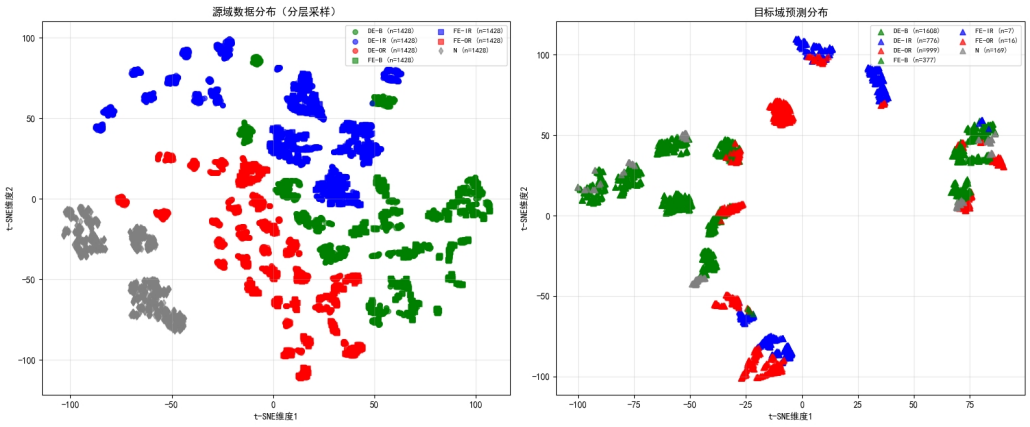


图 23　知识迁移可视化：源域知识成功迁移至目标域

7.3.2 约束机制的量化分析

本文设计的约束机制通过域差异最小化 (MMD)、类别平衡约束与特征一致性约束，在迁移过程中起到了关键的引导作用, 各约束项的数学表达式详见公式 (6.35)至(6.37) 该机制有效缩小了域间分布差异，确保了故障类别的平衡迁移，并维护了同类故障特征在跨域条件下的一致性，显著提升了诊断模型的泛化能力。

如图 24所示，四个子图分别展示了约束机制的作用过程：

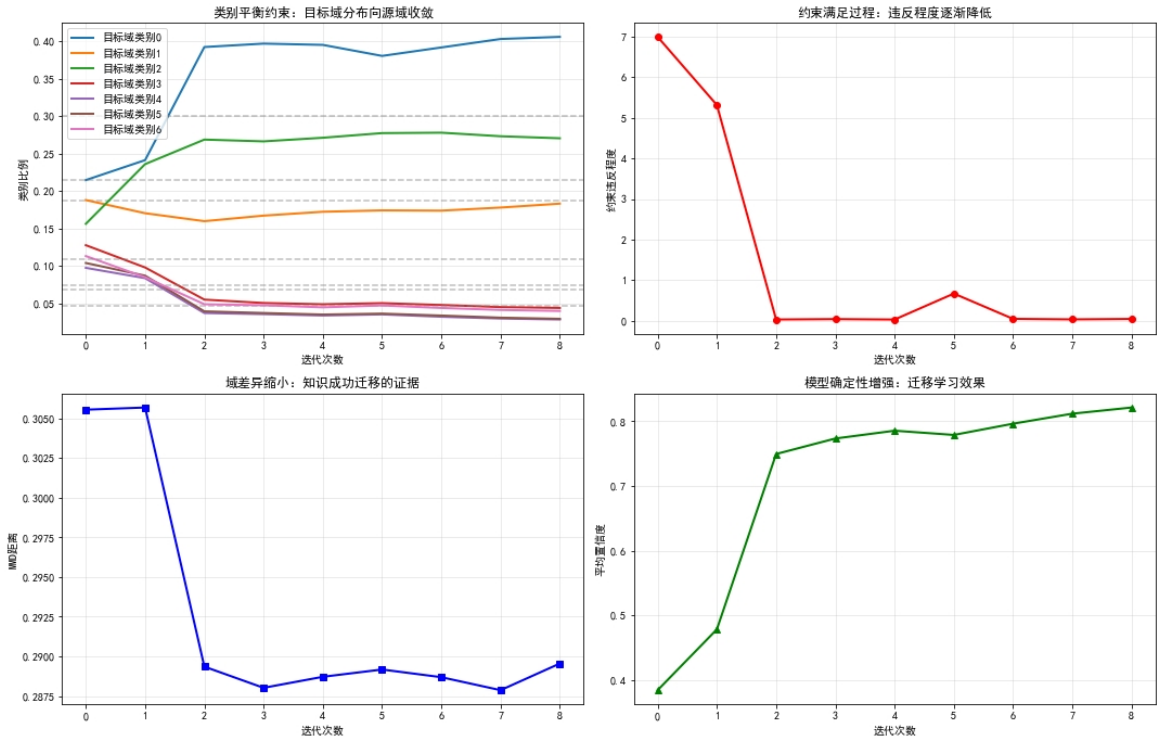


图 24　约束机制的可解释性分析

从图中可以观察到：

1. **类别平衡约束（左上）**：目标域各类别比例（实线）逐渐向源域分布（虚线）收敛，在第 2 次迭代后基本稳定；

2. **约束违反程度（右上）**：从初始的 7.0 快速下降至 0.04 以下，表明约束得到了有效的满足；

3. **MMD 距离变化（左下）**：MMD 距离从 0.3055 降至 0.2896，域差异减少了 5.2%，证明了源域知识成功迁移到了目标域；

4. **置信度提升（右下）**：平均置信度从 0.384 提升至 0.821，表明模型对目标域的诊断确定性显著增强。

7.4 事后可解释性分析

7.4.1 决策依据追溯

事后可解释性可借助 XGBoost 算法的可解释性，分析诊断模型的决策路径，反推诊断结果的依据。在上述问题 3 的目标域诊断中，对于每个目标域样本 x_i^t，其诊断决策过程可分解为如下公式：

$$P(y_i = c|x_i^t) = \sigma\left(\sum_{k=1}^{K} f_k(x_i^t)\right)_c \tag{7.42}$$

其中，$\sigma(\cdot)_c$ 为 softmax 函数的第 c 个输出。

通过追踪样本在每棵决策树中的路径，可以识别关键决策节点：

$$\mathcal{P}_i^k = \{(v_t, \tau_t, d_t)|t \in \text{path}(x_i, T_k)\} \tag{7.43}$$

其中，v_t 为节点 t 使用的特征，τ_t 为分裂阈值，d_t 为分裂方向。

7.4.2 故障机理关联

基于轴承故障机理，本文建立了诊断结果与故障特征物理的关联映射。以外圈故障为例，当目标域样本故障类型被诊断为外圈故障时，可由以下公式验证：

$$\text{Confidence}_{OR} = \omega_1 \cdot A(f_{BPFO}) + \omega_2 \cdot R_{periodic} + \omega_3 \cdot E_{impulse} \tag{7.44}$$

其中，$A(f_{BPFO})$ 为故障特征频率处的幅值，$R_{periodic}$ 为周期性指标，$E_{impulse}$ 为冲击能量。

7.4.3 置信度分析与可靠性评估

本文中诊断模型输出的置信度反映了故障诊断的可靠程度。本文定义的置信度校准误差（ECE）计算公式如下：

$$\text{ECE} = \sum_{m=1}^{M} \frac{|B_m|}{n} |\text{acc}(B_m) - \text{conf}(B_m)| \tag{7.45}$$

其中，B_m 为第 m 个置信度区间的样本集合，$\text{acc}(B_m)$ 和 $\text{conf}(B_m)$ 分别为该区间的准确率和平均置信度。

根据最终的实验结果，得到了目标域 16 个文件的诊断置信度分布如下：

1. 高置信度（>0.8）：11 个文件（68.8%），包括 B、C、D、E 等；
2. 中置信度（0.6-0.8）：2 个文件（12.5%），包括 I 和 L；
3. 低置信度（<0.6）：3 个文件（18.8%），包括 A、K 和 M。

7.5 综合可解释性框架

基于上述分析，本文以事前可解释性 $\mathcal{I}_{\text{prior}}$、迁移过程可解释性 $\mathcal{I}_{\text{transfer}}$ 与事后可解释性 $\mathcal{I}_{\text{post}}$ 构建了三层次的可解释性框架，系统覆盖了模型从前期准备、迁移过程到结果解释的全过程可解释性 $\mathcal{I}_{\text{total}}$。可解释性框架如下：

$$\mathcal{I}_{total} = \mathcal{I}_{prior} + \mathcal{I}_{transfer} + \mathcal{I}_{post} \tag{7.46}$$

其中：

$$\mathcal{I}_{prior} = \{f_{\text{structure}}, f_{\text{feature}}, f_{\text{mechanism}}\} \tag{7.47}$$

$$\mathcal{I}_{transfer} = \{f_{\text{alignment}}, f_{\text{constraint}}, f_{\text{adaptation}}\} \tag{7.48}$$

$$\mathcal{I}_{post} = \{f_{\text{decision}}, f_{\text{confidence}}, f_{\text{validation}}\} \tag{7.49}$$

各函数的含义如下表 11所示。

表 11　算法参数说明

符号	含义
$f_{\text{structure}}$	模型结构可解释性分析（如网络架构、模块组成等）
f_{feature}	特征的可解释性（物理含义，特征提取）
$f_{\text{mechanism}}$	任务内在机制的可解释性依据（如轴承故障机理）
f_{aligned}	知识迁移性解释（如源域-目标域特征/分布的对齐方式)
$f_{\text{constraint}}$	迁移过程的约束条件
$f_{\text{adaptation}}$	模型对目标域的自适应机制
f_{decision}	剖析决策路径，解释决策原因
$f_{\text{confidence}}$	量化的结果置信度
$f_{\text{validation}}$	外部验证模型合理性

7.6 问题四小结与分析

综上所述，本文对事前、迁移过程和事后三个维度的迁移诊断模型的可解释性分析结果如下：

1. **事前可解释性**：采用 XGBoost 树模型确保了模型结构的透明性，通过特征重要性的量化揭示出关键的诊断特征；

2. **迁移过程可解释性**：通过 t-SNE 可视化展示了知识迁移路径，且约束机制的量化分析（MMD 降低 5.2%、置信度提升 113.5%）验证了迁移效果；

3. **事后可解释性**：通过结合故障机理和追溯决策路径，建立了诊断结果与物理特征的映射关系，置信度分析表明 68.8% 的诊断结果具有高可靠性。

该可解释性框架不仅提高了诊断人员对模型的信任度，也为后续的模型优化和工程应用提供了理论支撑。

8 模型评价

8.1 模型优点

（1）本文提出算法创新的 CTL-ABC 约束迁移学习模型，通过引入类别平衡约束和特征一致性约束，有效解决了源域与目标域分布差异问题。相比于无约束的 UTL-ABC 算法，CTL-ABC 在 MMD 距离（0.2886 vs 0.2962）、平均置信度（0.865 vs 0.853）等关键指标上均表现得更优秀，证明了基于 CTL-ABC 约束机制的有效性。

（2）本文构建了事前-迁移过程-事后三个的层次可解释性框架，通过 XGBoost 模型、t-SNE 可视化、约束机制量化分析等方法，使得“黑箱”模型的决策过程可追溯、可理解，显著提升了模型的实际应用价值与用户的信任度。

（3）本文采用了五种经典分类算法进行了综合的对比，XGBoost 在准确率、F1 分数等指标上均达到 0.999 的优异性能。同时通过 ROC 曲线、混淆矩阵等多维度的评价指标，全面验证了 CTL-ABC 约束迁移学习模型的有效性和鲁棒性。

8.2 模型缺点

由于 CTL-ABC 涉及多个超参数（如 λ_1、λ_2、α_1、α_2、ε、δ 等），而参数设置的合理性直接影响诊断模型的迁移效果，但本文对参数敏感性分析不够充分，在不同的场景下可能需要重新进行调优。

附录 A　参考文献

1. 顾润卫, 王孝铖, 汪成, 姜扬帆, 何成. 大型数控龙门铣轴承故障诊断系统设计 [J]. 机械工程师, 2025(09): 166-168+172.
2. 苑宇, 郭琦. 基于 EEMD 包络谱和 JS-SDAE 的轴承故障诊断 [J]. 大连交通大学学报, 2025, 46(04): 49-56+82. DOI: 10.13291/j.cnki.djdxac.2025.04.005.
3. 苏泓臣, 杨辰昕, 边豪杰, 于佳鑫, 张宇宁. 基于自适应经验小波变换的轴承故障诊断研究 [J]. 核科学与工程, 2025, 45(03): 510-518. DOI: 10.20190/j.cnki.02580918.202503017.
4. 田嘉野, 梁朋飞, 袁晓明, 张立杰. 变工况不平衡条件下的滚动轴承故障诊断 [J]. 机械设计与制造, 2025(06): 332-337. DOI: 10.19356/j.cnki.1001-3997.2025.06.007.
5. 刘鹏. 基于深度学习和时频域特征融合的滚动轴承故障诊断研究 [D]. 内蒙古科技大学, 2025.
6. 于洪池, 崔咪咪. 多源传感迁移学习的河道重金属溯源监测进展 [J]. 实验与分析, 2025, 3(03): 71-75. DOI: 10.20175/j.syyfx.20250307.
7. 付强, 易学良, 符栋梁. 基于声-振信号融合的滚动轴承故障诊断 [J]. 自动化应用, 2025, 66(17): 190-195. DOI: 10.19769/j.zdhy.2025.17.052.
8. 邱骏驹, 张继波. 可解释性深度学习模型在网络入侵检测中的应用与优化 [J]. 网络安全技术与应用, 2025(07): 30-33.
9. 李辉. 约束与非约束联合的机器学习地震相分析方法研究与应用 [D]. 成都理工大学, 2022.
10. 第 5 章打破” 计算黑箱”，可解释人工智能构建下一代人工智能通用范式 [J]. 下一代创新科技, 2024(00): 41-52.
11. 杨文阳, 杨益慧. 聚焦可解释性：知识追踪模型综述与展望 [J]. 现代教育技术, 2024, 34(05): 53-63.
12. 任家琦, 王继烨. 基于迁移学习的配电网高阻接地故障特征提取与定位方法 [J]. 家电维修, 2025(09): 128-130.
13. 贡莹莹. 基于深度迁移学习的轴承故障诊断研究 [D]. 安徽理工大学, 2025.
14. 郑云亮, 娄月新, 白贵德, 陈诺, 赵杰超. 基于不平衡样本的电机滚动轴承故障智能诊断方法 [J]. 船舶工程, 2025, 47(S1): 620-628. DOI: 10.13788/j.cnki.cbgc.2025.S1.101.
15. 杨珊珊. 基于深度学习特征空间的可迁移对抗样本攻击技术研究 [D]. 北京邮电大学, 2023.
16. Maridu Bhargavi, Sivadi Balakrishna. Transfer learning based hybrid feature learning framework for enhanced skin cancer diagnosis using deep feature integration[J]. Engineering Science and Technology, an International Journal, 2025, 69: 102135.

附录

下面仅展示部分代码，具体的其它代码见附件 1

代码一：UTL-ABC 算法训练.py

```
import numpy as np
import pandas as pd
import pickle
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics.pairwise import rbf_kernel
from sklearn.manifold import TSNE
from sklearn.decomposition import PCA
from sklearn.preprocessing import LabelEncoder, StandardScaler
import xgboost as xgb
from scipy.optimize import minimize
from scipy.special import softmax
import warnings
warnings.filterwarnings('ignore')

# 设置中文字体
plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False

# ==============================================================================

# 数据加载和预处理（保持不变）
# ==============================================================================

class DataLoader:
    """数据加载器"""

    def __init__(self, model_path=None, scaler_path=None,
                 encoder_path=None):
        """
␣␣␣␣␣␣␣␣初始化

␣␣␣␣␣␣␣␣Parameters:
␣␣␣␣␣␣␣␣----------
␣␣␣␣␣␣␣␣model_path␣:␣str,␣XGBoost模型路径
```

```
␣␣␣␣␣␣␣␣scaler_path␣:␣str,␣标准化器路径
␣␣␣␣␣␣␣␣encoder_path␣:␣str,␣标签编码器路径
␣␣␣␣␣␣␣␣"""
        self.model_path = model_path
        self.scaler_path = scaler_path
        self.encoder_path = encoder_path

        # 初始化工具
        self.xgb_model = None
        self.scaler = None
        self.label_encoder = None

        # 尝试加载预训练模型和工具
        if model_path:
            self.load_pretrained_tools()
        else:
            print("未指定模型路径，将创建新的编码器和标准化器")
            self.label_encoder = LabelEncoder()
            self.scaler = StandardScaler()

    def load_pretrained_tools(self):
        """加载预训练的模型和工具"""
        # 加载XGBoost模型
        if self.model_path:
            try:
                with open(self.model_path, 'rb') as f:
                    self.xgb_model = pickle.load(f)
                print(f"□␣成功加载XGBoost模型:␣{self.model_path}")
            except Exception as e:
                print(f"□␣加载XGBoost模型失败:␣{e}")
                self.xgb_model = None

        # 加载标准化器
        if self.scaler_path:
            try:
                with open(self.scaler_path, 'rb') as f:
                    loaded_scaler = pickle.load(f)
                    # 检查是否是StandardScaler对象
                    if hasattr(loaded_scaler, 'transform'):
                        self.scaler = loaded_scaler
                        print(f"□␣成功加载标准化器:␣{self.
                            scaler_path}")
```

```
                    else:
                        print(f"□␣标准化器格式不正确，创建新的标准化器")
                        self.scaler = StandardScaler()
            except Exception as e:
                print(f"□␣加载标准化器失败:␣{e}")
                self.scaler = StandardScaler()
        else:
            self.scaler = StandardScaler()

        # 加载标签编码器
        if self.encoder_path:
            try:
                with open(self.encoder_path, 'rb') as f:
                    loaded_encoder = pickle.load(f)
                    # 检查加载的对象类型
                    if isinstance(loaded_encoder, LabelEncoder):
                        self.label_encoder = loaded_encoder
                        print(f"□␣成功加载标签编码器:␣{self.encoder_path}")
                    elif isinstance(loaded_encoder, (list, np.ndarray)):
                        # 如果是类别列表，创建新的编码器
                        print(f"□␣检测到类别列表，创建新的标签编码器")
                        self.label_encoder = LabelEncoder()
                        self.label_encoder.classes_ = np.array(loaded_encoder)
                    else:
                        print(f"□␣标签编码器格式不正确:␣{type(loaded_encoder)}")
                        self.label_encoder = LabelEncoder()
            except Exception as e:
                print(f"□␣加载标签编码器失败:␣{e}")
                self.label_encoder = LabelEncoder()
        else:
            self.label_encoder = LabelEncoder()

    def load_data(self, source_path, target_path):
        """
␣␣␣␣␣␣␣␣加载源域和目标域数据

```

```
        Parameters:
        ----------
        source_path : str, 源域数据路径
        target_path : str, 目标域数据路径

        Returns:
        --------
        X_source, y_source, X_target, target_info
        """
        print("\n加载数据集...")

        # 加载源域数据
        try:
            source_df = pd.read_csv(source_path)
            print(f"源域数据形状: {source_df.shape}")
        except Exception as e:
            print(f"□ 无法加载源域数据: {e}")
            return None, None, None, None

        # 提取源域特征和标签
        feature_cols = [f'PC{i}' for i in range(1, 14)]
        X_source = source_df[feature_cols].values
        y_source_labels = source_df['label1'].values

        # 打印原始标签分布
        unique_labels = np.unique(y_source_labels)
        print(f"源域标签类别: {unique_labels}")
        label_counts = pd.Series(y_source_labels).value_counts()
        print(f"源域类别分布:\n{label_counts}")

        # 编码标签
        try:
            # 如果编码器已经训练过
            if hasattr(self.label_encoder, 'classes_') and len(
                self.label_encoder.classes_) > 0:
                # 检查是否所有标签都能被编码
                unknown_labels = set(y_source_labels) - set(self.
                    label_encoder.classes_)
                if unknown_labels:
                    print(f"□ 发现未知标签: {unknown_labels}, 重新
                        训练编码器")
                    self.label_encoder = LabelEncoder()
```

```
                    y_source = self.label_encoder.fit_transform(
                        y_source_labels)
                else:
                    y_source = self.label_encoder.transform(
                        y_source_labels)
            else:
                # 训练新的编码器
                print("训练新的标签编码器...")
                y_source = self.label_encoder.fit_transform(
                    y_source_labels)

            print(f"编码后的标签范围:␣{np.min(y_source)}␣-␣{np.max
                (y_source)}")
            print(f"标签映射:␣{dict(zip(self.label_encoder.
                classes_,␣range(len(self.label_encoder.classes_))))
                }")

        except Exception as e:
            print(f"□␣标签编码失败:␣{e}")
            print("使用简单的标签映射...")
            # 创建简单的标签映射
            label_map = {label: i for i, label in enumerate(
                unique_labels)}
            y_source = np.array([label_map[label] for label in
                y_source_labels])

            # 更新编码器
            self.label_encoder = LabelEncoder()
            self.label_encoder.classes_ = unique_labels

        # 加载目标域数据
        try:
            target_df = pd.read_csv(target_path)
            print(f"\n目标域数据形状:␣{target_df.shape}")
        except Exception as e:
            print(f"□␣无法加载目标域数据:␣{e}")
            return X_source, y_source, None, None

        X_target = target_df[feature_cols].values
        target_files = target_df['file_label'].values

        # 统计目标域文件分布
```

```
        file_counts = pd.Series(target_files).value_counts()
        print(f"目标域文件分布:\n{file_counts}")

        # 数据标准化
        if self.scaler is not None:
            try:
                # 检查标准化器是否已经训练
                if hasattr(self.scaler, 'mean_') and self.scaler.
                   mean_ is not None:
                    print("\n使用已训练的标准化器...")
                    X_source = self.scaler.transform(X_source)
                    X_target = self.scaler.transform(X_target)
                else:
                    print("\n训练新的标准化器...")
                    X_source = self.scaler.fit_transform(X_source)
                    X_target = self.scaler.transform(X_target)
                print("□␣数据标准化完成")
            except Exception as e:
                print(f"□␣标准化失败:␣{e}, 使用原始数据")

        return X_source, y_source, X_target, target_files

#
    ================================================================================

# 无约束迁移学习核心算法
#
    ================================================================================

class UnconstrainedTransferLearning:
    """无约束的迁移学习"""

    def __init__(self, base_model=None, label_encoder=None):
        """
␣␣␣␣␣␣␣␣初始化

␣␣␣␣␣␣␣␣Parameters:
␣␣␣␣␣␣␣␣----------
␣␣␣␣␣␣␣␣base_model␣:␣预训练的模型（可选）
␣␣␣␣␣␣␣␣label_encoder␣:␣标签编码器（可选）
␣␣␣␣␣␣␣␣"""
        self.base_model = base_model
```

```
        self.label_encoder = label_encoder if label_encoder is not
            None else LabelEncoder()

        # 获取类别数
        if hasattr(self.label_encoder, 'classes_'):
            self.n_classes = len(self.label_encoder.classes_)
        else:
            self.n_classes = 4  # 默认4个类别

        # 超参数
        self.lambda1 = 0.3  # MMD权重
        self.lambda2 = 0.5  # 伪标签权重
        self.lambda3 = 0.1  # 正则化权重

        # 记录优化历史
        self.history = {
            'loss': [],
            'mmd': [],
            'pseudo_labels': [],
            'confidence': [],
            'label_change_rate': []
        }

    def compute_mmd(self, X_source, X_target, gamma=1.0):
        """计算最大均值差异(MMD)"""
        n_s = len(X_source)
        n_t = len(X_target)

        # 为了计算效率，采样部分数据
        max_samples = 500
        if n_s > max_samples:
            idx_s = np.random.choice(n_s, max_samples, replace=
                False)
            X_source = X_source[idx_s]
            n_s = max_samples
        if n_t > max_samples:
            idx_t = np.random.choice(n_t, max_samples, replace=
                False)
            X_target = X_target[idx_t]
            n_t = max_samples

        # 计算核矩阵
```

```
259         K_ss = rbf_kernel(X_source, X_source, gamma=gamma)
260         K_tt = rbf_kernel(X_target, X_target, gamma=gamma)
261         K_st = rbf_kernel(X_source, X_target, gamma=gamma)
262 
263         # MMD计算
264         mmd = np.sum(K_ss)/(n_s**2) + np.sum(K_tt)/(n_t**2) - 2*np
                .sum(K_st)/(n_s*n_t)
265 
266         return np.sqrt(max(0, mmd))
267 
268     def coral_alignment(self, X_source, X_target, lambda_reg=0.01)
            :
269         """CORAL特征对齐"""
270         d = X_source.shape[1]
271 
272         # 计算协方差矩阵
273         C_s = np.cov(X_source.T) + lambda_reg * np.eye(d)
274         C_t = np.cov(X_target.T) + lambda_reg * np.eye(d)
275 
276         try:
277             # 使用SVD分解计算矩阵平方根
278             U_s, S_s, V_s = np.linalg.svd(C_s)
279             C_s_sqrt_inv = U_s @ np.diag(1/np.sqrt(S_s + 1e-5)) @
                    V_s
280 
281             U_t, S_t, V_t = np.linalg.svd(C_t)
282             C_t_sqrt = U_t @ np.diag(np.sqrt(S_t + 1e-5)) @ V_t
283 
284             # 对齐变换
285             X_source_aligned = (X_source - np.mean(X_source, axis
                    =0)) @ C_s_sqrt_inv @ C_t_sqrt
286             X_source_aligned += np.mean(X_target, axis=0)
287 
288         except Exception as e:
289             print(f"␣CORAL对齐失败:␣{e}, 使用简化方法")
290             # 简化的对齐方法
291             X_source_aligned = (X_source - np.mean(X_source, axis
                    =0)) / (np.std(X_source, axis=0) + 1e-6)
292             X_source_aligned = X_source_aligned * np.std(X_target,
                     axis=0) + np.mean(X_target, axis=0)
293 
294         return X_source_aligned, X_target
```

```
295
296     def iterative_optimization(self, X_source, y_source, X_target,
            max_iter=10, confidence_threshold=0.7):
297         """迭代优化主流程（无约束版本）"""
298         print("\n" + "="*60)
299         print("开始无约束迁移学习优化")
300         print("="*60)
301
302         # 计算源域类别分布（仅用于监控，不作为约束）
303         unique_classes = np.unique(y_source)
304         self.n_classes = len(unique_classes)
305         source_class_dist = np.bincount(y_source, minlength=self.
               n_classes) / len(y_source)
306         print(f"源域类别分布:␣{source_class_dist}")
307
308         # 初始化
309         X_s_aligned = X_source.copy()
310         X_t_aligned = X_target.copy()
311         prev_labels = None
312
313         # 如果没有基础模型，训练一个
314         if self.base_model is None:
315             print("训练初始XGBoost模型...")
316             self.base_model = xgb.XGBClassifier(
317                 n_estimators=50,
318                 max_depth=3,
319                 learning_rate=0.1,
320                 random_state=42,
321                 n_jobs=-1
322             )
323             # 使用部分数据训练以加速
324             train_size = min(5000, len(X_source))
325             idx = np.random.choice(len(X_source), train_size,
                    replace=False)
326             self.base_model.fit(X_source[idx], y_source[idx])
327             print("□␣初始模型训练完成")
328
329         for iteration in range(max_iter):
330             print(f"\n---␣迭代␣{iteration+1}/{max_iter}␣---")
331
332             # Step 1: 计算MMD
333             mmd = self.compute_mmd(X_s_aligned, X_t_aligned)
```

```
            self.history['mmd'].append(mmd)
            print(f"MMD距离: {mmd:.4f}")

            # Step 2: CORAL特征对齐
            if iteration > 0:
                X_s_aligned, X_t_aligned = self.coral_alignment(
                    X_source, X_target)

            # Step 3: 生成伪标签和置信度
            try:
                pseudo_probs = self.base_model.predict_proba(
                    X_t_aligned)
                pseudo_labels = np.argmax(pseudo_probs, axis=1)
                confidence = np.max(pseudo_probs, axis=1)
            except Exception as e:
                print(f"□预测失败: {e}, 使用随机预测")
                # 模拟预测
                pseudo_probs = softmax(np.random.randn(len(
                    X_target), self.n_classes), axis=1)
                pseudo_labels = np.argmax(pseudo_probs, axis=1)
                confidence = np.max(pseudo_probs, axis=1)

            self.history['pseudo_labels'].append(pseudo_labels)
            self.history['confidence'].append(confidence)

            # 监控目标域预测分布（不作为约束）
            target_dist = np.mean(pseudo_probs, axis=0)
            print(f"目标域预测分布: {target_dist.round(3)}")

            # Step 4: 选择高置信度样本
            high_conf_mask = confidence > confidence_threshold
            n_high_conf = np.sum(high_conf_mask)
            print(f"高置信度样本数: {n_high_conf}/{len(X_target)}
                ({n_high_conf/len(X_target)*100:.1f}%)")
            print(f"平均置信度: {np.mean(confidence):.3f}")

            # Step 5: 更新模型
            if n_high_conf > 10:  # 至少有10个高置信度样本
                # 构建增强训练集
                # 采样源域数据以平衡训练集大小
                n_source_sample = min(2000, len(X_source))
                idx_source = np.random.choice(len(X_source),
```

```
                n_source_sample, replace=False)

            X_train = np.vstack([X_s_aligned[idx_source],
                X_t_aligned[high_conf_mask]])
            y_train = np.hstack([y_source[idx_source],
                pseudo_labels[high_conf_mask]])

            # 样本权重（仅基于置信度，无约束调整）
            source_weights = np.ones(n_source_sample)
            target_weights = confidence[high_conf_mask]
            weights = np.hstack([source_weights,
                target_weights])

            # 重新训练模型
            self.base_model = xgb.XGBClassifier(
                n_estimators=50,
                max_depth=3,
                learning_rate=0.1,
                random_state=42,
                n_jobs=-1
            )
            self.base_model.fit(X_train, y_train,
                sample_weight=weights)
            print("□␣模型更新完成")

        # Step 6: 计算总损失（简化版，无约束项）
        total_loss = self.lambda1 * mmd
        self.history['loss'].append(total_loss)
        print(f"总损失:␣{total_loss:.4f}")

        # Step 7: 收敛检查
        if prev_labels is not None:
            label_changes = np.sum(pseudo_labels !=
                prev_labels)
            change_rate = label_changes / len(pseudo_labels)
            self.history['label_change_rate'].append(
                change_rate)
            print(f"标签变化率:␣{change_rate:.2%}")

            if change_rate < 0.01:
                print("\n□␣模型已收敛！")
                break
```

```
407
408             prev_labels = pseudo_labels.copy()
409
410             # 动态调整置信度阈值
411             if iteration > 3 and n_high_conf < len(X_target) *
                    0.3:
412                 confidence_threshold = max(0.5,
                        confidence_threshold - 0.05)
413                 print(f"调整置信度阈值为:␣{confidence_threshold:.2
                        f}")
414
415         return pseudo_labels, confidence
416
417 #
    ================================================================================
418 # 可视化模块（简化版）
419 #
    ================================================================================
420 def plot_results(X_source, y_source, X_target, predictions,
    confidence, history):
421     """生成结果可视化"""
422
423     fig = plt.figure(figsize=(15, 10))
424
425     # 图1: MMD变化
426     ax1 = plt.subplot(2, 3, 1)
427     ax1.plot(history['mmd'], 'o-', linewidth=2, markersize=6)
428     ax1.set_xlabel('迭代次数')
429     ax1.set_ylabel('MMD距离')
430     ax1.set_title('域差异变化')
431     ax1.grid(True, alpha=0.3)
432
433     # 图2: 标签变化率
434     if 'label_change_rate' in history and len(history['
            label_change_rate']) > 0:
435         ax2 = plt.subplot(2, 3, 2)
436         ax2.plot(history['label_change_rate'], 's-', color='red',
                linewidth=2, markersize=6)
437         ax2.set_xlabel('迭代次数')
438         ax2.set_ylabel('标签变化率')
```

```
        ax2.set_title('预测稳定性')
        ax2.grid(True, alpha=0.3)

    # 图3: 平均置信度
    ax3 = plt.subplot(2, 3, 3)
    avg_conf = [np.mean(c) for c in history['confidence']]
    ax3.plot(avg_conf, '^-', color='green', linewidth=2,
       markersize=6)
    ax3.set_xlabel('迭代次数')
    ax3.set_ylabel('平均置信度')
    ax3.set_title('预测置信度变化')
    ax3.grid(True, alpha=0.3)

    # 图4: 置信度分布
    ax4 = plt.subplot(2, 3, 4)
    ax4.hist(confidence, bins=30, edgecolor='black', alpha=0.7)
    ax4.axvline(x=0.7, color='r', linestyle='--', label='阈值')
    ax4.set_xlabel('置信度')
    ax4.set_ylabel('样本数')
    ax4.set_title('最终置信度分布')
    ax4.legend()
    ax4.grid(True, alpha=0.3)

    # 图5: 类别分布
    ax5 = plt.subplot(2, 3, 5)
    unique_pred, counts = np.unique(predictions, return_counts=
       True)
    ax5.bar(unique_pred, counts, color='skyblue', edgecolor='black
       ')
    ax5.set_xlabel('类别')
    ax5.set_ylabel('样本数')
    ax5.set_title('预测类别分布')
    ax5.grid(True, alpha=0.3)

    # 图6: 总损失
    if 'loss' in history and len(history['loss']) > 0:
        ax6 = plt.subplot(2, 3, 6)
        ax6.plot(history['loss'], 'd-', color='purple', linewidth
           =2, markersize=6)
        ax6.set_xlabel('迭代次数')
        ax6.set_ylabel('总损失')
        ax6.set_title('总损失变化')
```

```
        ax6.grid(True, alpha=0.3)

    plt.tight_layout()
    plt.show()

    return fig

# ============================================================================

# 主程序
# ============================================================================

def main():
    """主程序入口"""

    # 设置路径
    MODEL_PATH = r"E:\华为杯\saved_models\best_model_xgboost.pkl"
    SCALER_PATH = r"E:\华为杯\saved_models\scaler.pkl"
    ENCODER_PATH = r"E:\华为杯\saved_models\label_encoder.pkl"
    SOURCE_DATA_PATH = r"E:\华为杯\df2_pca_features.csv"
    TARGET_DATA_PATH = r"E:\华为杯\目标域划窗结果\
       target_domain_pca_13d修改后的.csv"

    print("="*60)
    print("无约束迁移学习系统")
    print("="*60)

    # Step 1: 创建数据加载器
    loader = DataLoader(MODEL_PATH, SCALER_PATH, ENCODER_PATH)

    # Step 2: 加载数据
    X_source, y_source, X_target, target_files = loader.load_data(
        SOURCE_DATA_PATH, TARGET_DATA_PATH
    )

    if X_source is None or X_target is None:
        print("␣数据加载失败，程序退出")
        return None, None

    # Step 3: 创建迁移学习器
```

```
    transfer_learner = UnconstrainedTransferLearning(
        base_model=loader.xgb_model,
        label_encoder=loader.label_encoder
    )

    # Step 4: 执行迁移学习
    predictions, confidence = transfer_learner.
       iterative_optimization(
        X_source, y_source, X_target,
        max_iter=50,
        confidence_threshold=0.7
    )

    # Step 5: 生成结果
    print("\n" + "="*60)
    print("预测结果汇总")
    print("="*60)

    # 反向编码标签
    try:
        if hasattr(loader.label_encoder, 'inverse_transform'):
            predicted_labels = loader.label_encoder.
               inverse_transform(predictions)
        else:
            predicted_labels = [f"Class_{i}" for i in predictions]
    except:
        predicted_labels = [f"Class_{i}" for i in predictions]

    # 按文件统计
    results_df = pd.DataFrame({
        'file': target_files,
        'prediction': predicted_labels,
        'confidence': confidence
    })

    # 按文件聚合
    file_predictions = {}
    unique_files = np.unique(target_files)

    for file in unique_files:
        file_data = results_df[results_df['file'] == file]
        # 使用众数作为最终预测
```

```
        pred_mode = file_data['prediction'].mode()[0] if len(
            file_data['prediction'].mode()) > 0 else file_data['
            prediction'].iloc[0]
        avg_conf = file_data['confidence'].mean()

        file_predictions[file] = {
            'prediction': pred_mode,
            'confidence': avg_conf,
            'n_samples': len(file_data)
        }

    # 输出结果
    print("\n最终预测结果（按文件）:")
    print("-"*60)
    print(f"{'文件':<10} {'预测标签':<15} {'置信度':<10} {'可靠性
        ':<10} {'样本数':<10}")
    print("-"*60)

    for file in sorted(file_predictions.keys()):
        result = file_predictions[file]
        reliability = '高' if result['confidence'] > 0.8 else '中'
            if result['confidence'] > 0.6 else '低'
        print(f"{file:<10} {result['prediction']:<15} {result['
            confidence']:.3f}      {reliability:<10} {result['
            n_samples']:<10}")

    # Step 6: 可视化
    print("\n生成可视化...")
    plot_results(X_source, y_source, X_target, predictions,
        confidence, transfer_learner.history)

    # Step 7: 保存结果
    output_df = pd.DataFrame(file_predictions).T
    output_df.to_csv('unconstrained_transfer_predictions.csv')
    print("\n□ 预测结果已保存到:
        unconstrained_transfer_predictions.csv")

    results_df.to_csv('unconstrained_transfer_detailed.csv', index
        =False)
    print("□ 详细结果已保存到:unconstrained_transfer_detailed.csv
        ")

```

```
    return file_predictions, transfer_learner.history

# 运行主程序
if __name__ == "__main__":
    predictions, history = main()

    if predictions is not None and history is not None:
        print("\n" + "="*60)
        print("迁移学习统计")
        print("="*60)

        if len(history['mmd']) > 0:
            print(f"初始MMD距离: {history['mmd'][0]:.4f}")
            print(f"最终MMD距离: {history['mmd'][-1]:.4f}")
            print(f"MMD减少比例: {(history['mmd'][0] - history['
                mmd'][-1])/history['mmd'][0]*100:.1f}%")

        if 'label_change_rate' in history and len(history['
            label_change_rate']) > 0:
            print(f"最终标签变化率: {history['label_change_rate
                '][-1]:.4f}")

        if len(history['confidence']) > 0:
            print(f"最终平均置信度: {np.mean(history['confidence
                '][-1]):.3f}")
```

代码二：CTL-ABC 算法训练.py

```
import numpy as np
import pandas as pd
import pickle
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics.pairwise import rbf_kernel
from sklearn.manifold import TSNE
from sklearn.decomposition import PCA
from sklearn.preprocessing import LabelEncoder, StandardScaler
import xgboost as xgb
from scipy.optimize import minimize
from scipy.special import softmax
import warnings
warnings.filterwarnings('ignore')
```

```
15
16  # 设置中文字体
17  plt.rcParams['font.sans-serif'] = ['SimHei']
18  plt.rcParams['axes.unicode_minus'] = False
19
20  #
        ============================================================================

21  # 数据加载和预处理（修复版）
22  #
        ============================================================================

23  class DataLoader:
24      """数据加载器"""
25
26      def __init__(self, model_path=None, scaler_path=None,
            encoder_path=None):
27          """
28  ␣␣␣␣␣␣␣␣初始化
29
30  ␣␣␣␣␣␣␣␣Parameters:
31  ␣␣␣␣␣␣␣␣----------
32  ␣␣␣␣␣␣␣␣model_path␣:␣str,␣XGBoost模型路径
33  ␣␣␣␣␣␣␣␣scaler_path␣:␣str,␣标准化器路径
34  ␣␣␣␣␣␣␣␣encoder_path␣:␣str,␣标签编码器路径
35  ␣␣␣␣␣␣␣␣"""
36          self.model_path = model_path
37          self.scaler_path = scaler_path
38          self.encoder_path = encoder_path
39
40          # 初始化工具
41          self.xgb_model = None
42          self.scaler = None
43          self.label_encoder = None
44
45          # 尝试加载预训练模型和工具
46          if model_path:
47              self.load_pretrained_tools()
48          else:
49              print("未指定模型路径，将创建新的编码器和标准化器")
50              self.label_encoder = LabelEncoder()
51              self.scaler = StandardScaler()
```

```
52
53      def load_pretrained_tools(self):
54          """加载预训练的模型和工具"""
55          # 加载XGBoost模型
56          if self.model_path:
57              try:
58                  with open(self.model_path, 'rb') as f:
59                      self.xgb_model = pickle.load(f)
60                  print(f"□␣成功加载XGBoost模型:␣{self.model_path}")
61              except Exception as e:
62                  print(f"□␣加载XGBoost模型失败:␣{e}")
63                  self.xgb_model = None
64
65          # 加载标准化器
66          if self.scaler_path:
67              try:
68                  with open(self.scaler_path, 'rb') as f:
69                      loaded_scaler = pickle.load(f)
70                      # 检查是否是StandardScaler对象
71                      if hasattr(loaded_scaler, 'transform'):
72                          self.scaler = loaded_scaler
73                          print(f"□␣成功加载标准化器:␣{self.
                                scaler_path}")
74                      else:
75                          print(f"□␣标准化器格式不正确，创建新的标准
                                化器")
76                          self.scaler = StandardScaler()
77              except Exception as e:
78                  print(f"□␣加载标准化器失败:␣{e}")
79                  self.scaler = StandardScaler()
80          else:
81              self.scaler = StandardScaler()
82
83          # 加载标签编码器
84          if self.encoder_path:
85              try:
86                  with open(self.encoder_path, 'rb') as f:
87                      loaded_encoder = pickle.load(f)
88                      # 检查加载的对象类型
89                      if isinstance(loaded_encoder, LabelEncoder):
90                          self.label_encoder = loaded_encoder
91                          print(f"□␣成功加载标签编码器:␣{self.
```

```
                            encoder_path}")
92                      elif isinstance(loaded_encoder, (list, np.
                            ndarray)):
93                          # 如果是类别列表，创建新的编码器
94                          print(f"□␣检测到类别列表，创建新的标签编码
                                器")
95                          self.label_encoder = LabelEncoder()
96                          self.label_encoder.classes_ = np.array(
                                loaded_encoder)
97                      else:
98                          print(f"□␣标签编码器格式不正确:␣{type(
                                loaded_encoder)}")
99                          self.label_encoder = LabelEncoder()
100             except Exception as e:
101                 print(f"□␣加载标签编码器失败:␣{e}")
102                 self.label_encoder = LabelEncoder()
103         else:
104             self.label_encoder = LabelEncoder()
105
106     def load_data(self, source_path, target_path):
107         """
108 ␣␣␣␣␣␣␣␣加载源域和目标域数据
109
110 ␣␣␣␣␣␣␣␣Parameters:
111 ␣␣␣␣␣␣␣␣----------
112 ␣␣␣␣␣␣␣␣source_path␣:␣str,␣源域数据路径
113 ␣␣␣␣␣␣␣␣target_path␣:␣str,␣目标域数据路径
114
115 ␣␣␣␣␣␣␣␣Returns:
116 ␣␣␣␣␣␣␣␣--------
117 ␣␣␣␣␣␣␣␣X_source,␣y_source,␣X_target,␣target_info
118 ␣␣␣␣␣␣␣␣"""
119         print("\n加载数据集...")
120
121         # 加载源域数据
122         try:
123             source_df = pd.read_csv(source_path)
124             print(f"源域数据形状:␣{source_df.shape}")
125         except Exception as e:
126             print(f"□␣无法加载源域数据:␣{e}")
127             return None, None, None, None
128
```

```
        # 提取源域特征和标签
        feature_cols = [f'PC{i}' for i in range(1, 14)]
        X_source = source_df[feature_cols].values
        y_source_labels = source_df['label1'].values

        # 打印原始标签分布
        unique_labels = np.unique(y_source_labels)
        print(f"源域标签类别: {unique_labels}")
        label_counts = pd.Series(y_source_labels).value_counts()
        print(f"源域类别分布:\n{label_counts}")

        # 编码标签
        try:
            # 如果编码器已经训练过
            if hasattr(self.label_encoder, 'classes_') and len(
                self.label_encoder.classes_) > 0:
                # 检查是否所有标签都能被编码
                unknown_labels = set(y_source_labels) - set(self.
                    label_encoder.classes_)
                if unknown_labels:
                    print(f"  发现未知标签: {unknown_labels}, 重新
                        训练编码器")
                    self.label_encoder = LabelEncoder()
                    y_source = self.label_encoder.fit_transform(
                        y_source_labels)
                else:
                    y_source = self.label_encoder.transform(
                        y_source_labels)
            else:
                # 训练新的编码器
                print("训练新的标签编码器...")
                y_source = self.label_encoder.fit_transform(
                    y_source_labels)

            print(f"编码后的标签范围: {np.min(y_source)} - {np.max
                (y_source)}")
            print(f"标签映射: {dict(zip(self.label_encoder.
                classes_, range(len(self.label_encoder.classes_))))
                }")

        except Exception as e:
            print(f"  标签编码失败: {e}")
```

```
            print("使用简单的标签映射...")
            # 创建简单的标签映射
            label_map = {label: i for i, label in enumerate(
                unique_labels)}
            y_source = np.array([label_map[label] for label in
                y_source_labels])

            # 更新编码器
            self.label_encoder = LabelEncoder()
            self.label_encoder.classes_ = unique_labels

        # 加载目标域数据
        try:
            target_df = pd.read_csv(target_path)
            print(f"\n目标域数据形状:␣{target_df.shape}")
        except Exception as e:
            print(f"□␣无法加载目标域数据:␣{e}")
            return X_source, y_source, None, None

        X_target = target_df[feature_cols].values
        target_files = target_df['file_label'].values

        # 统计目标域文件分布
        file_counts = pd.Series(target_files).value_counts()
        print(f"目标域文件分布:\n{file_counts}")

        # 数据标准化
        if self.scaler is not None:
            try:
                # 检查标准化器是否已经训练
                if hasattr(self.scaler, 'mean_') and self.scaler.
                    mean_ is not None:
                    print("\n使用已训练的标准化器...")
                    X_source = self.scaler.transform(X_source)
                    X_target = self.scaler.transform(X_target)
                else:
                    print("\n训练新的标准化器...")
                    X_source = self.scaler.fit_transform(X_source)
                    X_target = self.scaler.transform(X_target)
                print("□␣数据标准化完成")
            except Exception as e:
                print(f"□␣标准化失败:␣{e}, 使用原始数据")
```

```
201
202         return X_source, y_source, X_target, target_files
203
204 #
    ============================================================================

205 # 迁移学习核心算法（保持不变）
206 #
    ============================================================================

207 class ConstrainedTransferLearning:
208     """带约束的迁移学习"""
209
210     def __init__(self, base_model=None, label_encoder=None,
        epsilon=0.15, delta=1.0):
211         """
212 ␣␣␣␣␣␣␣␣初始化
213
214 ␣␣␣␣␣␣␣␣Parameters:
215 ␣␣␣␣␣␣␣␣----------
216 ␣␣␣␣␣␣␣␣base_model␣:␣预训练的模型（可选）
217 ␣␣␣␣␣␣␣␣label_encoder␣:␣标签编码器（可选）
218 ␣␣␣␣␣␣␣␣epsilon␣:␣float,␣类别平衡约束的容忍度
219 ␣␣␣␣␣␣␣␣delta␣:␣float,␣特征一致性约束的阈值
220 ␣␣␣␣␣␣␣␣"""
221         self.base_model = base_model
222         self.label_encoder = label_encoder if label_encoder is not
            None else LabelEncoder()
223         self.epsilon = epsilon
224         self.delta = delta
225
226         # 获取类别数
227         if hasattr(self.label_encoder, 'classes_'):
228             self.n_classes = len(self.label_encoder.classes_)
229         else:
230             self.n_classes = 4  # 默认4个类别
231
232         # 超参数
233         self.lambda1 = 0.3  # MMD权重
234         self.lambda2 = 0.5  # 伪标签权重
235         self.lambda3 = 0.1  # 正则化权重
236         self.alpha1 = 1.0   # 类别平衡约束权重
```

```
        self.alpha2 = 0.5   # 特征一致性约束权重

        # 记录优化历史
        self.history = {
            'loss': [],
            'mmd': [],
            'constraint_violation': [],
            'pseudo_labels': [],
            'confidence': []
        }

    def compute_mmd(self, X_source, X_target, gamma=1.0):
        """计算最大均值差异(MMD)"""
        n_s = len(X_source)
        n_t = len(X_target)

        # 为了计算效率，采样部分数据
        max_samples = 500
        if n_s > max_samples:
            idx_s = np.random.choice(n_s, max_samples, replace=
                False)
            X_source = X_source[idx_s]
            n_s = max_samples
        if n_t > max_samples:
            idx_t = np.random.choice(n_t, max_samples, replace=
                False)
            X_target = X_target[idx_t]
            n_t = max_samples

        # 计算核矩阵
        K_ss = rbf_kernel(X_source, X_source, gamma=gamma)
        K_tt = rbf_kernel(X_target, X_target, gamma=gamma)
        K_st = rbf_kernel(X_source, X_target, gamma=gamma)

        # MMD计算
        mmd = np.sum(K_ss)/(n_s**2) + np.sum(K_tt)/(n_t**2) - 2*np
            .sum(K_st)/(n_s*n_t)

        return np.sqrt(max(0, mmd))

    def coral_alignment(self, X_source, X_target, lambda_reg=0.01)
        :
```

```
        """CORAL特征对齐"""
        d = X_source.shape[1]

        # 计算协方差矩阵
        C_s = np.cov(X_source.T) + lambda_reg * np.eye(d)
        C_t = np.cov(X_target.T) + lambda_reg * np.eye(d)

        try:
            # 使用SVD分解计算矩阵平方根
            U_s, S_s, V_s = np.linalg.svd(C_s)
            C_s_sqrt_inv = U_s @ np.diag(1/np.sqrt(S_s + 1e-5)) @
               V_s

            U_t, S_t, V_t = np.linalg.svd(C_t)
            C_t_sqrt = U_t @ np.diag(np.sqrt(S_t + 1e-5)) @ V_t

            # 对齐变换
            X_source_aligned = (X_source - np.mean(X_source, axis
               =0)) @ C_s_sqrt_inv @ C_t_sqrt
            X_source_aligned += np.mean(X_target, axis=0)

        except Exception as e:
            print(f"␣CORAL对齐失败:␣{e},␣使用简化方法")
            # 简化的对齐方法
            X_source_aligned = (X_source - np.mean(X_source, axis
               =0)) / (np.std(X_source, axis=0) + 1e-6)
            X_source_aligned = X_source_aligned * np.std(X_target,
                axis=0) + np.mean(X_target, axis=0)

        return X_source_aligned, X_target

    def compute_class_balance_constraint(self, pseudo_probs,
       source_class_dist):
        """计算类别平衡约束违反程度"""
        # 计算目标域的预测分布
        target_dist = np.mean(pseudo_probs, axis=0)

        # 计算违反程度
        violation = 0
        for k in range(min(len(target_dist), len(source_class_dist
           ))):
            diff = abs(target_dist[k] - source_class_dist[k])
```

```
            if diff > self.epsilon:
                violation += (diff - self.epsilon) ** 2

        return violation, target_dist

    def compute_feature_consistency_constraint(self, X_source,
       y_source, X_target, pseudo_labels):
        """计算特征一致性约束违反程度"""
        violation = 0

        # 对每个类别计算违反程度
        for k in range(self.n_classes):
            # 源域中该类别的样本
            source_k = X_source[y_source == k]
            # 目标域中预测为该类别的样本
            target_k = X_target[pseudo_labels == k]

            if len(source_k) > 0 and len(target_k) > 0:
                # 计算类中心
                source_center = np.mean(source_k, axis=0)
                target_center = np.mean(target_k, axis=0)

                # 计算中心距离
                dist = np.linalg.norm(source_center -
                   target_center)
                if dist > self.delta:
                    violation += (dist - self.delta) ** 2

        return violation

    def iterative_optimization(self, X_source, y_source, X_target,
        max_iter=10, confidence_threshold=0.7):
        """迭代优化主流程"""
        print("\n" + "="*60)
        print("开始带约束的迁移学习优化")
        print("="*60)

        # 计算源域类别分布
        unique_classes = np.unique(y_source)
        self.n_classes = len(unique_classes)
        source_class_dist = np.bincount(y_source, minlength=self.
           n_classes) / len(y_source)
```

```
        print(f"源域类别分布:␣{source_class_dist}")

        # 初始化
        X_s_aligned = X_source.copy()
        X_t_aligned = X_target.copy()
        prev_labels = None

        # 如果没有基础模型，训练一个
        if self.base_model is None:
            print("训练初始XGBoost模型...")
            self.base_model = xgb.XGBClassifier(
                n_estimators=50,
                max_depth=3,
                learning_rate=0.1,
                random_state=42,
                n_jobs=-1
            )
            # 使用部分数据训练以加速
            train_size = min(5000, len(X_source))
            idx = np.random.choice(len(X_source), train_size,
               replace=False)
            self.base_model.fit(X_source[idx], y_source[idx])
            print("□␣初始模型训练完成")

        for iteration in range(max_iter):
            print(f"\n---␣迭代␣{iteration+1}/{max_iter}␣---")

            # Step 1: 计算MMD
            mmd = self.compute_mmd(X_s_aligned, X_t_aligned)
            self.history['mmd'].append(mmd)
            print(f"MMD距离:␣{mmd:.4f}")

            # Step 2: CORAL特征对齐
            if iteration > 0:
                X_s_aligned, X_t_aligned = self.coral_alignment(
                   X_source, X_target)

            # Step 3: 生成伪标签和置信度
            try:
                pseudo_probs = self.base_model.predict_proba(
                   X_t_aligned)
                pseudo_labels = np.argmax(pseudo_probs, axis=1)
```

```
                confidence = np.max(pseudo_probs, axis=1)
            except Exception as e:
                print(f"□␣预测失败:␣{e}, 使用随机预测")
                # 模拟预测
                pseudo_probs = softmax(np.random.randn(len(
                    X_target), self.n_classes), axis=1)
                pseudo_labels = np.argmax(pseudo_probs, axis=1)
                confidence = np.max(pseudo_probs, axis=1)

            self.history['pseudo_labels'].append(pseudo_labels)
            self.history['confidence'].append(confidence)

            # Step 4: 计算约束违反程度
            balance_violation, target_dist = self.
                compute_class_balance_constraint(
                pseudo_probs, source_class_dist
            )
            print(f"目标域预测分布:␣{target_dist.round(3)}")
            print(f"类别平衡约束违反:␣{balance_violation:.4f}")

            consistency_violation = self.
                compute_feature_consistency_constraint(
                X_s_aligned, y_source, X_t_aligned, pseudo_labels
            )
            print(f"特征一致性约束违反:␣{consistency_violation:.4f
                }")

            self.history['constraint_violation'].append(
                balance_violation + consistency_violation
            )

            # Step 5: 选择高置信度样本
            high_conf_mask = confidence > confidence_threshold
            n_high_conf = np.sum(high_conf_mask)
            print(f"高置信度样本数:␣{n_high_conf}/{len(X_target)}␣
                ({n_high_conf/len(X_target)*100:.1f}%)")
            print(f"平均置信度:␣{np.mean(confidence):.3f}")

            # Step 6: 更新模型
            if n_high_conf > 10:  # 至少有10个高置信度样本
                # 构建增强训练集
                # 采样源域数据以平衡训练集大小
```

```
                n_source_sample = min(2000, len(X_source))
                idx_source = np.random.choice(len(X_source),
                    n_source_sample, replace=False)

                X_train = np.vstack([X_s_aligned[idx_source],
                    X_t_aligned[high_conf_mask]])
                y_train = np.hstack([y_source[idx_source],
                    pseudo_labels[high_conf_mask]])

                # 样本权重
                source_weights = np.ones(n_source_sample)
                target_weights = confidence[high_conf_mask]

                # 根据约束调整权重
                if balance_violation > 0.01:
                    for k in range(self.n_classes):
                        if k < len(target_dist) and k < len(
                            source_class_dist):
                            if target_dist[k] > source_class_dist[
                                k] + self.epsilon:
                                mask = pseudo_labels[
                                    high_conf_mask] == k
                                target_weights[mask] *= 0.8
                            elif target_dist[k] <
                                source_class_dist[k] - self.epsilon
                                :
                                mask = pseudo_labels[
                                    high_conf_mask] == k
                                target_weights[mask] *= 1.2

                weights = np.hstack([source_weights,
                    target_weights])

                # 重新训练模型
                self.base_model = xgb.XGBClassifier(
                    n_estimators=50,
                    max_depth=3,
                    learning_rate=0.1,
                    random_state=42,
                    n_jobs=-1
                )
                self.base_model.fit(X_train, y_train,
```

```
                    sample_weight=weights)

            # Step 7: 计算总损失
            total_loss = (self.lambda1 * mmd +
                          self.alpha1 * balance_violation +
                          self.alpha2 * consistency_violation)
            self.history['loss'].append(total_loss)
            print(f"总损失: {total_loss:.4f}")

            # Step 8: 收敛检查
            if prev_labels is not None:
                label_changes = np.sum(pseudo_labels !=
                    prev_labels)
                change_rate = label_changes / len(pseudo_labels)
                print(f"标签变化率: {change_rate:.2%}")

                if change_rate < 0.01 and balance_violation < 0.1:
                    print("\n□ 模型已收敛! ")
                    break

            prev_labels = pseudo_labels.copy()

            # 动态调整置信度阈值
            if iteration > 3 and n_high_conf < len(X_target) *
                0.5:
                confidence_threshold = max(0.5,
                    confidence_threshold - 0.05)
                print(f"调整置信度阈值为: {confidence_threshold:.2
                    f}")

        return pseudo_labels, confidence

#
    ================================================================================

# 可视化模块(简化版)
#
    ================================================================================

def plot_results(X_source, y_source, X_target, predictions,
    confidence, history):
    """生成结果可视化"""
```

```
489
490     fig = plt.figure(figsize=(15, 10))
491
492     # 图1: MMD变化
493     ax1 = plt.subplot(2, 3, 1)
494     ax1.plot(history['mmd'], 'o-', linewidth=2, markersize=6)
495     ax1.set_xlabel('迭代次数')
496     ax1.set_ylabel('MMD距离')
497     ax1.set_title('域差异变化')
498     ax1.grid(True, alpha=0.3)
499
500     # 图2: 约束违反
501     ax2 = plt.subplot(2, 3, 2)
502     ax2.plot(history['constraint_violation'], 's-', color='red',
        linewidth=2, markersize=6)
503     ax2.set_xlabel('迭代次数')
504     ax2.set_ylabel('约束违反程度')
505     ax2.set_title('约束满足情况')
506     ax2.grid(True, alpha=0.3)
507
508     # 图3: 平均置信度
509     ax3 = plt.subplot(2, 3, 3)
510     avg_conf = [np.mean(c) for c in history['confidence']]
511     ax3.plot(avg_conf, '^-', color='green', linewidth=2,
        markersize=6)
512     ax3.set_xlabel('迭代次数')
513     ax3.set_ylabel('平均置信度')
514     ax3.set_title('预测置信度变化')
515     ax3.grid(True, alpha=0.3)
516
517     # 图4: 置信度分布
518     ax4 = plt.subplot(2, 3, 4)
519     ax4.hist(confidence, bins=30, edgecolor='black', alpha=0.7)
520     ax4.axvline(x=0.7, color='r', linestyle='--', label='阈值')
521     ax4.set_xlabel('置信度')
522     ax4.set_ylabel('样本数')
523     ax4.set_title('最终置信度分布')
524     ax4.legend()
525     ax4.grid(True, alpha=0.3)
526
527     # 图5: 类别分布
528     ax5 = plt.subplot(2, 3, 5)
```

```
    unique_pred, counts = np.unique(predictions, return_counts=True)
    ax5.bar(unique_pred, counts, color='skyblue', edgecolor='black')
    ax5.set_xlabel('类别')
    ax5.set_ylabel('样本数')
    ax5.set_title('预测类别分布')
    ax5.grid(True, alpha=0.3)

    # 图6: 总损失
    if 'loss' in history and len(history['loss']) > 0:
        ax6 = plt.subplot(2, 3, 6)
        ax6.plot(history['loss'], 'd-', color='purple', linewidth=2, markersize=6)
        ax6.set_xlabel('迭代次数')
        ax6.set_ylabel('总损失')
        ax6.set_title('总损失变化')
        ax6.grid(True, alpha=0.3)

    plt.tight_layout()
    plt.show()

    return fig

#
    ================================================================================

# 主程序
#
    ================================================================================

def main():
    MODEL_PATH = r"E:\华为杯\saved_models\best_model_xgboost.pkl"
    SCALER_PATH = r"E:\华为杯\saved_models\scaler.pkl"
    ENCODER_PATH = r"E:\华为杯\saved_models\label_encoder.pkl"
    SOURCE_DATA_PATH = r"E:\华为杯\df2_pca_features.csv"
    TARGET_DATA_PATH = r"E:\华为杯\目标域划窗结果\target_domain_pca_13d修改后的.csv"

    print("="*60)
    print("带约束的迁移学习系统")
    print("="*60)
```

```
563
564     # Step 1: 创建数据加载器
565     loader = DataLoader(MODEL_PATH, SCALER_PATH, ENCODER_PATH)
566
567     # Step 2: 加载数据
568     X_source, y_source, X_target, target_files = loader.load_data(
569         SOURCE_DATA_PATH, TARGET_DATA_PATH
570     )
571
572     if X_source is None or X_target is None:
573         print("□␣数据加载失败，程序退出")
574         return None, None
575
576     # Step 3: 创建迁移学习器
577     transfer_learner = ConstrainedTransferLearning(
578         base_model=loader.xgb_model,
579         label_encoder=loader.label_encoder,
580         epsilon=0.15,
581         delta=1.0
582     )
583
584     # Step 4: 执行迁移学习
585     predictions, confidence = transfer_learner.
        iterative_optimization(
586         X_source, y_source, X_target,
587         max_iter=50,
588         confidence_threshold=0.7
589     )
590
591     # Step 5: 生成结果
592     print("\n" + "="*60)
593     print("预测结果汇总")
594     print("="*60)
595
596     # 反向编码标签
597     try:
598         if hasattr(loader.label_encoder, 'inverse_transform'):
599             predicted_labels = loader.label_encoder.
                inverse_transform(predictions)
600         else:
601             predicted_labels = [f"Class_{i}" for i in predictions]
602     except:
```

```
603         predicted_labels = [f"Class_{i}" for i in predictions]
604
605     # 按文件统计
606     results_df = pd.DataFrame({
607         'file': target_files,
608         'prediction': predicted_labels,
609         'confidence': confidence
610     })
611
612     # 按文件聚合
613     file_predictions = {}
614     unique_files = np.unique(target_files)
615
616     for file in unique_files:
617         file_data = results_df[results_df['file'] == file]
618         # 使用众数作为最终预测
619         pred_mode = file_data['prediction'].mode()[0] if len(
               file_data['prediction'].mode()) > 0 else file_data['
               prediction'].iloc[0]
620         avg_conf = file_data['confidence'].mean()
621
622         file_predictions[file] = {
623             'prediction': pred_mode,
624             'confidence': avg_conf,
625             'n_samples': len(file_data)
626         }
627
628     # 输出结果
629     print("\n最终预测结果（按文件）:")
630     print("-"*60)
631     print(f"{'文件':<10}␣{'预测标签':<15}␣{'置信度':<10}␣{'可靠性
           ':<10}␣{'样本数':<10}")
632     print("-"*60)
633
634     for file in sorted(file_predictions.keys()):
635         result = file_predictions[file]
636         reliability = '高' if result['confidence'] > 0.8 else '中'
               if result['confidence'] > 0.6 else '低'
637         print(f"{file:<10}␣{result['prediction']:<15}␣{result['
               confidence']:.3f}␣␣␣␣␣␣{reliability:<10}␣{result['
               n_samples']:<10}")
638
```

```
    # Step 6: 可视化
    print("\n生成可视化...")
    plot_results(X_source, y_source, X_target, predictions,
        confidence, transfer_learner.history)

    # Step 7: 保存结果
    output_df = pd.DataFrame(file_predictions).T
    output_df.to_csv('transfer_learning_predictions.csv')
    print("\n□␣预测结果已保存到:␣transfer_learning_predictions.csv
        ")

    results_df.to_csv('transfer_learning_detailed.csv', index=
        False)
    print("□␣详细结果已保存到:␣transfer_learning_detailed.csv")

    return file_predictions, transfer_learner.history

# 运行主程序
if __name__ == "__main__":
    predictions, history = main()

    if predictions is not None and history is not None:
        print("\n" + "="*60)
        print("迁移学习统计")
        print("="*60)

        if len(history['mmd']) > 0:
            print(f"初始MMD距离:␣{history['mmd'][0]:.4f}")
            print(f"最终MMD距离:␣{history['mmd'][-1]:.4f}")
            print(f"MMD减少比例:␣{(history['mmd'][0]␣-␣history['
                mmd'][-1])/history['mmd'][0]*100:.1f}%")

        if len(history['constraint_violation']) > 0:
            print(f"最终约束违反:␣{history['constraint_violation
                '][-1]:.4f}")

        if len(history['confidence']) > 0:
            print(f"最终平均置信度:␣{np.mean(history['confidence
                '][-1]):.3f}")
```